



上海交通大学学位论文

VLA 模型在昇腾算力上的适配与优化

姓 名：陈启炜

学 号：522031910299

导 师：古金宇

学 院：计算机学院

专业名称：软件工程

申请学位层次：学士

2026 年 5 月

A Dissertation Submitted to
Shanghai Jiao Tong University for the Degree of Bachelor

ADAPTATION AND OPTIMIZATION OF
VISION-LANGUAGE-ACTION MODELS ON
ASCEND COMPUTING PLATFORMS

Author: Qiwei Chen

Supervisor: Jinyu Gu

School of Computer Science
Shanghai Jiao Tong University
Shanghai, P.R. China

May, 2026

上海交通大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，已在文中以适当方式予以致谢。若在论文撰写过程中使用了人工智能工具，本人已遵循《上海交通大学关于在教育教学中使用 AI 的规范》，确保人工智能生成内容的应用场景、引用范围及标注方式均符合规定，并杜绝学术不端行为。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期：2026 年 5 月 25 日 陈启炜

上海交通大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☒ 公开论文

☐ 内部论文，保密 ☐ 1 年 / ☐ 2 年 / ☐ 3 年，过保密期后适用本授权书。

☐ 秘密论文，保密 ____ 年（不超过 10 年），过保密期后适用本授权书。

☐ 机密论文，保密 ____ 年（不超过 20 年），过保密期后适用本授权书。

（请在以上方框内选择打“√”）

学位论文作者签名：陈启炜

日期：2026 年 5 月 25 日

指导教师签名：古金宇

日期：2026 年 5 月 25 日

摘 要

视觉-语言-动作模型 (Vision-Language-Action Model, VLA) 统一建模视觉、语言和动作, 是具身智能策略学习的重要框架。现有 VLA 开源实现主要面向 CUDA 生态, 而端侧机器人控制要求低时延、低网络依赖和受限功耗下的稳定推理。针对国产端侧部署和连续动作生成延迟问题, 本文构建多类型 VLA 与具身策略模型在 Ascend 310P 上的适配与集成验证流程, 并面向以 π_0 、 $\pi_{0.5}$ 和 SmolVLA 为代表的视觉语言模型 (Vision-Language Model) + 动作专家模块 (Action Expert) 类模型研究推理加速。

平台适配流程覆盖动态图推理、静态图推理加速、混合精度优化、统一推理接口和仿真任务接入, 并按模型可运行、输出数值一致与任务成功率一致三层检查迁移结果。在 6 组模型适配案例中, 静态图优化后的平均推理延迟由 1049.80 ms 降至 360.15 ms; 在已验证的 LIBERO、ALOHA 和 PushT 代表性仿真任务中, Ascend 310P 与 NVIDIA GPU 参考运行的任务成功率保持一致, 输出动作向量的余弦相似度高于 99.00%。

推理加速方面, 本文提出锚点残差去噪 (Anchor-Residual Denoising, ARD) 方法, 将原始多步去噪改写为少量全量去噪锚点调用和轻量残差纠偏。ARD 复用动作专家模块锚点速度场外推少步动作轨迹, 并由轻量纠偏模型补偿其与全量去噪教师轨迹之间的偏差。该方法以原始速度场作为方向先验, 使小模型专注于局部残差; 同时结合视觉语言模型前缀上下文压缩和回合级成功样本加权, 使纠偏目标更贴近闭环任务效果。

实验显示, SmolVLA 加权双锚点纠偏在 LIBERO 4 个任务集上的平均成功率为 78.0%, 高于原始全量去噪模型的 74.3%; $\pi_{0.5}$ 单锚点残差纠偏平均成功率为 89.8%, 接近原始模型的 91.1%。在 Ascend 310P、AGX Orin 与 AGX Thor 的动态图动作块口径下, ARD 相对原始多步模型实现 1.44× 至 4.12× 端到端加速; 最终实现 Ascend 310P 上 SmolVLA 和 $\pi_{0.5}$ 最终动作块单次推理延迟分别为 80.123 ms 和 240.384 ms。结果表明, 面向国产端侧算力的 VLA 适配与 ARD 推理加速具有可行性。

关键词: 视觉-语言-动作模型, 昇腾平台, 去噪蒸馏, 残差纠偏, 推理加速

ABSTRACT

Vision-Language-Action (VLA) models jointly model vision, language, and robot actions for embodied policy learning. Most open-source VLA implementations target the CUDA ecosystem, while edge robot control requires stable low-latency inference under limited power and network dependence. This thesis builds an Ascend 310P adaptation and integration validation workflow for multiple VLA and embodied policy models, and studies inference acceleration for Vision-Language Model + Action Expert models represented by π_0 , $\pi_{0.5}$, and SmolVLA.

For platform adaptation, the workflow covers dynamic graph inference, static graph inference acceleration, mixed precision optimization, a unified inference interface, and simulation task integration. It checks model executability, output numerical consistency, and task success rate consistency. Across six adaptation cases, static graph optimization reduces average inference latency from 1049.80 ms to 360.15 ms. On representative LIBERO, ALOHA, and PushT tasks, Ascend 310P matches the NVIDIA GPU reference in task success rate, and the cosine similarity of output action vectors is above 99.00%.

For acceleration, this thesis proposes ARD (Anchor-Residual Denoising), which rewrites multi step denoising as a few full denoising anchor calls plus lightweight residual correction. ARD reuses action expert anchor velocity fields to extrapolate short step action trajectories, and uses a lightweight correction model to compensate for their deviation from the full denoising teacher trajectory. The original velocity field provides a trajectory direction prior, while compressed vision-language prefix context and episode level success weighting better align residual learning with closed loop task performance.

Experiments show that SmolVLA weighted two anchor correction improves the LIBERO 4-suite average from 74.3% to 78.0%; $\pi_{0.5}$ single anchor correction reaches 89.8% versus the original 91.1%. Across Ascend 310P, AGX Orin, and AGX Thor dynamic graph action chunk tests, ARD gives 1.44 $\times$ to 4.12 $\times$ speedup. On Ascend 310P, the final static graph pipeline achieves single action chunk latencies of 80.123 ms for SmolVLA and 240.384 ms for $\pi_{0.5}$.

Key words: VLA Model, Ascend, ARD, Residual Correction, Inference Acceleration

目 录

摘 要 I

ABSTRACT II

第一章 绪论 1

 1.1 研究背景与意义 1

 1.2 国内外研究现状 3

 1.3 研究问题与挑战 4

 1.4 本文主要贡献 5

 1.5 论文结构安排 6

第二章 相关工作与背景知识 8

 2.1 视觉-语言-动作模型 8

 2.2 推理加速方法 9

 2.3 昇腾平台与 CANN 生态 10

 2.4 LIBERO 评测基准 11

 2.5 本章小结 11

第三章 VLA 模型昇腾平台适配 12

 3.1 适配目标与整体方案 12

 3.2 动态图适配 13

 3.3 静态图适配与优化 14

 3.4 适配结果与验证 16

 3.5 本章小结 18

第四章 锚点残差去噪推理加速方法 19

 4.1 VLA 推理时间分布 19

 4.2 问题形式化 22

 4.3 直接减步数的局限性 23

4.4	锚点残差去噪方案设计	24
4.4.1	设计观察：速度场相似性与累计偏移	24
4.4.2	整体方法	26
4.4.3	残差纠偏模型结构	27
4.4.4	训练数据构造与目标函数	28
4.5	两阶段锚点残差去噪	29
4.6	本章小结	30
第五章	实验评测	31
5.1	实验设置	31
5.2	直接减步数实验	32
5.3	SmoIVLA 残差纠偏消融实验	33
5.3.1	纠偏模型容量	33
5.3.2	单锚点与双锚点残差纠偏	33
5.3.3	加权训练	34
5.4	SmoIVLA 4 个任务集成功率评测	34
5.5	跨模型验证	36
5.6	跨随机种子稳定性复验	36
5.7	跨平台加速与昇腾验证	37
5.7.1	Ascend 310P 动态图延迟	37
5.7.2	端侧平台对比	38
5.7.3	昇腾静态图部署链路补充分析	38
5.8	分析与讨论	40
5.9	本章小结	41
第六章	总结与展望	42
6.1	主要工作总结	42
6.2	主要结论	43
6.3	局限性	44
6.4	未来工作	44
6.5	本章小结	45

参 考 文 献.....	46
附 录.....	49
1.1 主要符号与缩写.....	49
攻读学位期间学术论文和科研成果目录.....	53
致 谢.....	54

第一章 绪论

1.1 研究背景与意义

具身智能的目标是使智能体能够在物理环境中感知、理解并执行任务，相关中文综述也将大模型驱动的具身智能视为人工智能、机器人学和认知科学交叉融合的重要方向^[1-2]。随着大规模视觉语言模型的发展，机器人策略不再局限于针对单一任务训练的专用控制器，而是逐步转向能够同时理解视觉观测、语言指令和动作空间的视觉-语言-动作模型（Vision-Language-Action Model, VLA）。这类模型将语言中的任务意图、视觉中的物体与场景信息以及机器人自身状态统一到动作生成过程中，为桌面整理、抓取放置、双臂操作和长程任务执行提供了更通用的建模方式。近年来，RT-2、OpenVLA、 π_0 、 $\pi_{0.5}$ 和 SmolVLA 等工作表明，VLA 模型已经成为具身智能策略学习的重要技术路线^[3-7]。

与文本生成或图像生成任务不同，机器人策略模型的输出会直接作用于物理运动链路。推理延迟不仅影响用户感知到的响应速度，也会影响动作连续性、闭环控制频率和任务成功率。对于真实机器人系统，策略通常需要以稳定频率持续输出动作；若单次推理时间过长，控制链路就难以维持平滑的交互节奏，长程任务中的误差也更容易累积。面向移动操作机器人系统的发展研究同样指出，多模态感知、智能决策以及运动与操作联合规划是具身系统落地中的共性关键技术^[8]。因此，VLA 模型的实际落地不仅要求模型具备较强的语义理解和动作生成能力，还要求推理链路能够在端侧设备上稳定、高效地执行。

端侧部署对具身智能尤其重要。一方面，本地推理可以减少对网络链路和远端服务的依赖，降低通信延迟、网络抖动和数据传输风险；另一方面，移动机器人、机械臂工作站和边缘计算设备往往受到成本、功耗、体积和散热条件的限制，不能简单依靠更大规模的服务器级 GPU 来解决推理瓶颈。端智能推理加速综述也指出，端侧模型部署需要同时考虑模型压缩、软硬件协同优化、异构并行部署和大模型端侧优化等问题^[9]。近期针对 VLA 模型的跨 XPU 部署表征也指出，仅依赖桌面级 GPU 评测会掩盖异构边缘加速器在成本、能耗和时延之间的权衡空间^[10]。国产算力平台为端侧部署提供了新的硬件基础，但现有 VLA 开源实现大多围绕 CUDA 和通用 GPU 生态构建，直接迁移到 Ascend 310P 等平台时会遇到算子兼容、动态图执行、静态图编译和混合精度策略等工程问题。

因此，研究 VLA 模型在昇腾算力平台上的适配与优化具有两方面意义。首先，它能够为主流 VLA 与具身策略模型在国产端侧硬件上的部署提供可复用路径，降低模型迁移和验证成本。其次，它能够针对 VLA 推理链路中的主要时延瓶颈设计加速方法，在保持任务成功率的前提下降低动作生成延迟，从而提升具身智能系统的实际可部署性。本文围绕“能在昇腾平台上正确运行”和“能以更低延迟完成动作生成”两个目标展开研究，形成从平台适配到推理优化的完整技术链路。

图 1-1 概括了本文面向 VLA 模型端侧部署的主要问题和研究范围。左侧从端侧部署视角指出，代表性 VLA 模型迁移到 Ascend 310P 等端侧算力芯片时，通常面临工具链框架适配不足、算力与带宽受限以及推理延迟过高等问题；右侧给出本文关注的典型模型对象和优化方向，包括面向 ACT、 π_0 、 $\pi_{0.5}$ 、SmolVLA 等模型的昇腾适配，以及通过精度压缩、计算图优化、步数压缩和 KV 复用等方法构建轻量化模型与优化部署方案。

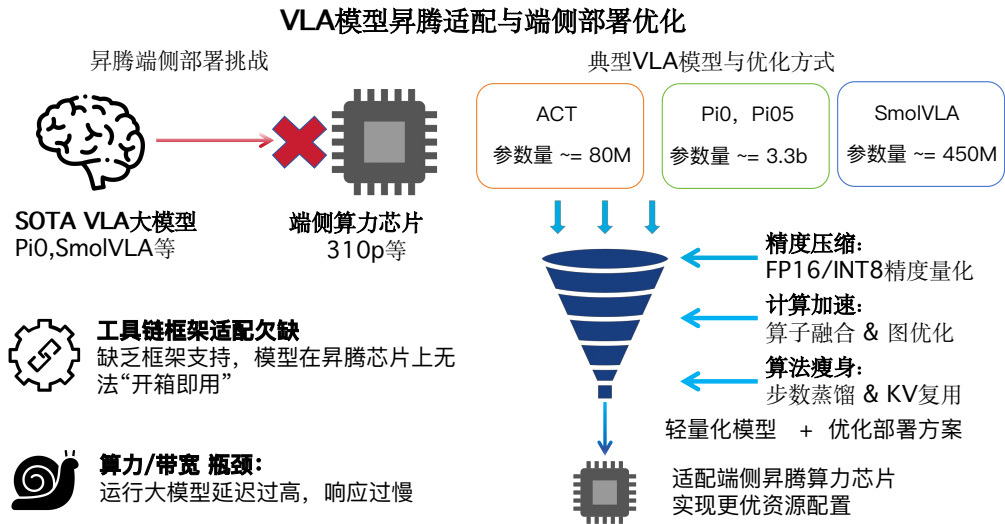


图 1-1 VLA 模型端侧部署挑战与本文研究范围。左侧概括主流 VLA 模型部署到端侧昇腾算力芯片时面临的工具链适配和算力瓶颈；右侧展示本文关注的典型模型规模与优化路径，目标是在端侧算力约束下实现更优的资源配置和推理效率。

1.2 国内外研究现状

现有 VLA 与具身策略模型可以按照动作生成方式大致分为三类^①。第一类是动作分块或行为克隆策略，例如 ACT 通过观测直接预测未来一段动作序列，具有结构清晰、推理路径较短的特点，适合用作轻量具身策略和平台适配对象。第二类是自回归动作词元 VLA，例如 RT-2 和 OpenVLA 将动作离散化或词元化，并在视觉语言模型框架下以自回归方式生成动作^[3-4]。第三类是扩散或 Flow Matching 连续动作策略，例如 Diffusion Policy、 π_0 、 $\pi_{0.5}$ 和 SmolVLA 等模型通过多步去噪或速度场预测生成连续动作^[5-7,11-12]。这三类方法在动作表示、推理过程和部署复杂度上存在明显差异，也对应不同的加速空间。

在连续动作 VLA 中，一类具有代表性的架构采用“视觉语言模型 (VLM) 负责语义理解，动作专家模块 (Action Expert, AE) 负责连续动作生成”的模块化方式。VLM 部分将语言指令和视觉观测编码为上下文条件，Action Expert 则在该条件下通过多步迭代生成动作块。这种结构有利于复用预训练视觉语言模型的语义能力，也便于在连续动作空间中表达平滑轨迹；但 Action Expert 的多步循环会带来额外推理时延，成为端侧部署中的重要瓶颈。本文的推理加速方法主要面向这一类 VLM + Action Expert 连续动作生成模型。

围绕生成模型推理加速，已有研究在扩散模型和 Flow Matching 模型中提出了少步采样、渐进蒸馏和一致性模型等方法，表明减少生成步数是降低推理延迟的重要方向^[12-13]。与此同时，大语言模型中的投机解码、EAGLE 系列方法，以及面向多模态 diffusion transformer 的 ASDSV 等工作，也说明可以利用轻量模型或近似步骤承担部分推理开销，再通过校验或纠偏机制减少主模型调用^[14-16]。

但是，上述方法不能直接套用于 VLA 连续动作生成场景。一方面，LLM 投机解码依赖离散词元的接受/拒绝校验，而 VLA 输出的是连续动作轨迹，动作是否有效往往需要通过闭环环境交互才能体现；ASDSV 等扩散投机验证方法虽然利用了连续去噪过程中的时间相关性，但其校验机制也难以直接迁移到 π_0 、SmolVLA 这类 VLA 模型结构中。另一方面，当前 VLA 模型的跨任务泛化能力仍有限，实际使用中常需要面向具体任务或数据分布进行微调；若直接采用完整学生模型或渐进蒸馏来替代全量去噪过程，通常需要较大的训练资源和任务数据成本。对于直接减少去噪步数的方法，VLA 场景中还存在任务成功率与推理频率之间的折中：激进减步会明显降低部

① 本段相关文献检索与初步筛选曾使用 DeepSeek R1 辅助，最终引用文献的真实性、相关性和表述均由作者人工核验。

分任务成功率，而保守减步带来的端到端延迟收益又较有限。因此，本文希望在保留原始 Action Expert 轨迹先验的基础上，通过轻量残差纠偏模型补偿少步外推误差，从而在尽量保持任务成功率的同时提高推理速度。

在国产算力适配方面，昇腾 CANN 软件栈提供了动态图执行、ATC 模型转换、OM 静态图推理和 ACL/ACLLite 运行时接口等工具链^[17-18]。这些工具为深度学习模型迁移到 Ascend NPU 提供了基础能力，但 VLA 模型通常同时包含视觉编码、语言编码、动作专家、动态张量形状和多模态预处理，直接转换时更容易暴露算子兼容和精度策略问题。当前公开资料中，针对通用分类、检测或语言模型的部署案例较多，而面向 VLA 与具身策略模型的系统性适配、验证和推理加速流程仍相对不足。

综上，已有研究分别推动了 VLA 模型能力、生成模型步数压缩和国产算力部署工具链的发展，但三者之间仍存在明显空缺：一方面，需要面向多类型 VLA 与具身策略模型建立可复用的 Ascend 310P 适配流程；另一方面，需要针对 VLM + Action Expert 架构中的多步动作生成瓶颈，设计既能降低延迟又能保持任务成功率的推理加速方法。

1.3 研究问题与挑战

本文围绕如下核心问题展开：如何使主流 VLA 与具身策略模型在国产昇腾端侧平台上实现可部署、正确且低延迟的推理，并进一步降低 VLA 模型推理开销。围绕这一问题，本文面临以下三个挑战。

挑战一：动态图效率、静态图转换与部署验证的协同适配。VLA 与具身策略模型结构差异较大，不同模型可能采用自回归动作词元、动作分块、扩散去噪或 Flow Matching 等不同生成方式，输入输出张量、动态张量形状、注意力结构和后处理逻辑差异明显。将这些模型迁移到 Ascend 310P 时，适配难点不仅在于使模型完成一次前向运行，而在于解决三类相互关联的问题。其一，动态图执行难以充分利用端侧 NPU 的硬件性能。多模态推理链路通常包含大量形状整理、索引更新、归一化和小规模张量算子，运行时容易产生频繁算子下发和 CPU-NPU 往返切换，使得硬件计算能力不能完全转化为端到端吞吐。其二，静态图转换工具链仍不够成熟。ONNX 导出和 ATC 编译过程中可能出现算子不支持、动态张量形状固化错误或 ONNX/OM 算子形状不一致等问题，而手工排查大规模计算图中的节点和张量形状差异耗时较高。其三，导出后的静态图缺乏充分的任务级校验机制。静态图一致性通常依赖模拟输入下的 PyTorch、ONNX 或 OM 输出对比，但实际部署中可能出现部分输入一致、部分输

入不一致的情况，仅靠少量离线样例难以保证闭环任务行为正确。

挑战二：小规模 Action Expert 的蒸馏收益约束。对于 VLM + Action Expert 类连续动作生成模型，VLM 负责将语言指令和视觉观测编码为语义条件，Action Expert 则在该条件下根据当前噪声状态迭代生成连续动作块。与 VLM 主干相比，Action Expert 的参数规模通常远小于语义编码部分：按本文使用的 LIBERO 模型权重主模块参数统计， π_0 的 VLM/Action Expert 约为 2.92B/575M， $\pi_{0.5}$ 约为 2.92B/691M，SmolVLA 约为 507M/97M。这种结构使加速方法不能简单依赖更大的学生模型替代原始 Action Expert，否则学生模型本身的开销和训练成本会削弱步数减少带来的收益；但若过度压缩，又会破坏动作轨迹并降低任务成功率。因此，本问题的关键约束是在充分利用现有 Action Expert 轨迹生成能力的同时减少多步去噪计算开销，使推理速度、动作质量和训练代价之间保持平衡。

挑战三：语义上下文规模与轻量纠偏目标的矛盾。连续动作是否正确不仅取决于当前动作噪声状态，还取决于语言指令、视觉观测、机器人状态和任务阶段。原始 VLM + Action Expert 架构本身已经通过 VLM 前缀 KV 缓存向 Action Expert 传递完整语义上下文；在本文使用的导出口径下， π_0 的 VLM 前缀 KV 缓存约包含 2.80M 个 fp16 元素，对应约 5.6 MB 数据，SmolVLA 的 VLM 前缀 KV 缓存约包含 3.62M 个 fp16 元素，对应约 7.2 MB 数据。若纠偏模型完全不使用上下文，就难以区分不同任务中的关键动作；但若轻量纠偏模型直接沿用完整 KV 缓存接口， N 层纠偏网络通常需要在每层读取并建模这一百万级上下文，KV 相关数据访问规模会近似随层数增长为 N 倍。例如 6 层结构仅完整 KV 读入量就约为 33.6 MB 至 43.2 MB，还不包括注意力中间激活、投影计算和归一化开销，从而削弱轻量化纠偏的收益。

1.4 本文主要贡献

针对上述问题，本文以 Ascend 310P 平台适配和 VLM + Action Expert 推理优化为主线，主要贡献如下。

第一，**多类型 VLA 的昇腾平台适配与验证。**完成了面向多类型主流 VLA 与具身策略模型的昇腾平台适配，并将适配代码、问题定位经验和验证流程整理后开源到 CANN 社区。适配工作覆盖动态图推理、静态图导出与编译、混合精度策略、算子兼容性修复和端到端正确性验证。正确性方面，在 π_0 的 ALOHA Transfer Cube、 $\pi_{0.5}$ 的 LIBERO 4 个任务集和 Diffusion Policy 的 PushT 等已验证任务上，Ascend 310P 与 GPU 参考平台的成功率分别保持为 91%、86% 和 90%，动作向量输出的余弦相似度

高于 99.00%，说明适配后的推理链路能够保持任务行为正确。性能方面，在 6 组模型适配案例中，静态图优化后的平均延迟由 1049.80 ms 降至 360.15 ms。以 π_0 动作块链路为例，Ascend 310P 静态图延迟为 350.30 ms，AGX Thor 官方 Docker 环境下的静态图延迟约为 163 ms；Thor 与 310P 的 FP16 峰值算力比为 $258/88 = 2.93$ ，而延迟比为 $350/163 = 2.15$ 。在该部署口径下，310P 与 Thor 的延迟差距小于峰值算力差距，体现出较好的相对延迟表现。

第二，**锚点残差去噪推理加速方法**。提出锚点残差去噪 (Anchor-Residual Denoising, ARD) 推理加速方法，用于降低 VLM + Action Expert 连续动作生成模型中的多步去噪开销。该方法首先执行少量锚点去噪，利用锚点速度给出后续去噪轨迹的方向先验；随后由轻量残差纠偏模型预测线性外推轨迹与全量去噪轨迹之间的残差。与直接替代 Action Expert 不同，该方法将学习目标约束在较小的残差空间中，从而在降低模型规模和调用次数的同时保留全量去噪模型的轨迹信息。

第三，**VLM 前缀上下文压缩与可配置锚点策略**。设计了 VLA 中面向 VLM 前缀的上下文压缩机制和可配置的去噪激进程度。残差纠偏模型通过压缩后的 VLM 前缀 KV 获取语言和视觉条件，避免直接沿用完整上下文接口带来的输入规模和数据搬运压力；推理过程中可通过单锚点或双锚点策略调节去噪压缩程度。该设计允许简单任务采用更激进的步数压缩，也允许复杂任务通过中间锚点缓解误差累积。

第四，**LIBERO 成功率与多平台延迟评估**。基于 LIBERO 基准系统评估了方法的任务成功率和推理延迟。成功率方面，在当前 LIBERO 4 个任务集平均口径下，SmolVLA 原始全量去噪模型、未加权双锚点纠偏方案和加权双锚点纠偏方案的成功率分别为 74.3%、74.5% 和 78.0%。延迟方面，在 Ascend 310P 动态图口径下， π_0 单锚点残差纠偏为 1.44 $\times$ ，SmolVLA 双锚点/单锚点 ARD 为 $2.00 \times / 2.43 \times$ ， $\pi_{0.5}$ 单锚点残差纠偏为 1.52 $\times$ ；同一 SmolVLA LIBERO 微调模型在 Ascend 310P、AGX Orin 与 AGX Thor 上的跨平台加速范围进一步达到 2.00 $\times$ 至 4.12 $\times$ 。综合跨模型与跨平台动态图实验，本文方法的端到端加速范围为 1.44 $\times$ 至 4.12 $\times$ 。

1.5 论文结构安排

本文共分为六章：第一章介绍研究背景、国内外研究现状、研究挑战与主要贡献；第二章梳理 VLA 模型、生成式动作策略推理加速、昇腾 CANN 生态和 LIBERO 评测基准；第三章给出 VLA 模型在 Ascend 310P 上的动态图与静态图适配方法；第四章介绍面向 VLM + Action Expert 连续动作生成模型的锚点残差去噪方法；第五章基于

LIBERO 4 个任务集成功率、跨模型验证、跨随机种子复验和多平台延迟测量对本文方法进行实验评估；第六章总结全文工作，分析困难任务上的方法局限，并讨论未来可继续研究的方向。

第二章 相关工作与背景知识

2.1 视觉-语言-动作模型

从技术实现看，VLA 及相关具身策略模型的差异主要体现在三个方面：动作空间是离散词元还是连续控制量，语义条件是通过通用 VLM 直接注入还是由任务数据间接学习，以及动作生成是否需要多步迭代。表 2-1 按模型结构和动作生成方式概括了本文涉及或对比的代表性模型，并从主要特点和局限性两个角度进行比较。其中，ACT 和 Diffusion Policy 更准确地说是具身策略或连续动作生成基线，而不是狭义的语言条件 VLA；本文将它们纳入讨论，是因为它们在动作表示和推理链路上与 VLA 部署问题直接相关^①。

表 2-1 代表性 VLA 与具身策略模型的技术特点

模型结构	主流模型示例	主要特点	局限性
动作分块行为克隆	ACT ^[19]	模型规模较小，训练和推理流程相对简单，可直接预测未来一段动作序列，适合快速构建模仿学习策略。	语义条件较弱，跨任务泛化能力主要依赖训练数据覆盖。
自回归动作词元 VLA	RT-2 ^[3] OpenVLA ^[4] OpenVLA-OFT ^[20]	将动作纳入 VLM/LLM 序列建模框架，能够复用视觉语言预训练知识；OpenVLA-OFT 进一步通过并行解码、动作分块和连续动作表示提升微调后策略效率。	逐词元解码会带来延迟压力；模型规模通常较大，跨任务泛化仍受机器人数据和微调分布影响。
扩散式连续动作策略	Diffusion Policy ^[11] RDT ^[21]	通过去噪过程建模多模态连续动作分布，适合精细操作和双臂任务，能够表达动作轨迹的不确定性。	需要多步去噪采样，推理延迟较大。
VLM + Action Expert	π_0 ^[5]	由 VLM 编码视觉语言条件，再由 Action Expert 生成连续动作，兼顾语义理解和连续控制。	Action Expert 仍需要多步去噪或速度场积分，端到端动作生成延迟较高。
Flow Matching	$\pi_{0.5}$ ^[6] SmolVLA ^[7]		

上述模型谱系说明，平台适配与推理优化关注的是不同层面的共性问题。在平台适配层面，动作分块、自回归词元、扩散和 Flow Matching 模型会分别暴露张量形状、注意力结构、循环调用和混合精度策略上的差异，因此需要覆盖多类型模型来验证迁

① 本章部分参考文献条目和格式整理曾使用 DeepSeek R1 辅助，作者对参考文献来源、真实性和相关性进行了人工校验。

移流程的可复用性。在推理优化层面, 本文优化方案主要针对 VLM + 动作专家模块 (Action Expert, AE) 连续动作生成结构的 VLA 模型: VLM 侧提供语言和视觉语义条件, Action Expert 侧在该条件下迭代更新动作噪声状态。近期跨 XPU 表征工作也观察到, VLA 推理常呈现 VLM 主干与 Action Expert 两阶段执行模式, 不同阶段的算力和访存特征会造成端侧硬件利用率差异^[10]。该结构的关键矛盾是, Action Expert 的参数规模通常小于 VLM 主干, 但其在每个动作块中重复执行, 因而成为端侧低延迟部署中更直接的可压缩对象。

2.2 推理加速方法

VLA 推理加速首先需要明确延迟来源。对于自回归动作词元模型, 延迟主要来自逐词元解码; 对于扩散或 Flow Matching 连续动作模型, 延迟主要来自多步去噪或速度场预测。Flow Matching 将生成过程表述为连续时间上的速度场学习问题, 推理时仍需要将连续轨迹离散为若干步进行数值积分^[12]; Rectified Flow 等工作进一步强调通过更直的概率路径降低采样难度^[22]。因此, 减少生成步数、降低每步模型调用成本, 或者复用相邻步的中间特征, 是生成式动作策略加速的自然方向。

在生成模型领域, 已有不少关于少步采样和步数蒸馏的研究用于降低扩散模型采样开销。Progressive Distillation 等方法通过让学生模型模拟教师模型多步采样的整体效果, 逐步减少推理步数^[13]; Consistency Models 则直接面向一步或少步生成建立一致性映射, 为少步采样提供了另一类训练目标^[23]。这类思想说明, 多步生成过程并非只能按原始步数执行, 若能够有效保持生成质量, 就可以用更少的模型调用获得接近教师模型的输出。对于 VLM + Action Expert 类 VLA 模型, 直接减少去噪步数同样可以降低 Action Expert 调用次数; 但 stepN 消融显示, 激进减少到 step1 时部分任务成功率明显下降, 而保留更多去噪步数来维持成功率时端到端速度收益又比较有限。因此, 本文需要解决的问题不是单纯减少步数, 而是在压缩 Action Expert 调用次数的同时尽量保持闭环任务成功率。

另一类相关思路是缓存复用和轻量模型辅助推理。大语言模型中的投机解码利用小模型生成候选词元, 再由大模型验证候选序列, 从而减少大模型的串行解码开销^[14]; 后续 SpecInfer、Medusa、EAGLE 及 EAGLE-3 等方法分别从词元树、多解码头、特征外推和多层特征融合等角度扩展了这一思路^[15,24-27]。这一范式为“用低成本模型承担部分推理工作”提供了启发, 但其校验机制依赖离散词元的接受/拒绝判断, 而 π_0 、SmolVLA 这类连续动作 VLA 模型没有可直接迁移的逐词元在线校验过程。

对于连续去噪模型，也已有工作尝试利用相邻时间步的相关性降低推理开销。ASDSV 面向多模态 diffusion transformer，将近似投机扩散与投机验证结合，用初末状态对齐近似验证多个投机步骤^[16]。在机器人策略方向，BAC 通过区块级自适应缓存复用 Diffusion Policy 中间特征^[28]，TS-DP 则将轨迹级草稿模型与强化学习调度器结合，用于动态调整 Diffusion Policy 的投机参数^[29]。这些工作共同说明，连续去噪过程中的时间相关性和中间特征冗余具有可利用价值；但它们或依赖特定去噪结构下的在线验证假设，或主要面向 Diffusion Policy 的缓存复用与调度。对于 VLM + Action Expert 架构，推理条件还包含 VLM 前缀上下文和任务相关微调后的动作生成分布，现有验证或缓存机制难以直接作为 π_0 、SmolVLA 等模型的通用加速方案。

基于上述背景，本文不将轻量模型设计为完整替代 Action Expert 的学生模型。一方面，当前 VLA 模型往往仍依赖具体任务或数据分布上的微调，完整蒸馏整个 Action Expert 去噪过程需要重新学习任务相关的动作生成映射，训练数据和算力成本较高；另一方面，原始 Action Expert 的少量锚点预测已经包含主要动作方向，可以作为低成本外推的基础。因此，本文将推理加速问题表述为“少步锚点去噪 + 残差纠偏”：复用原始 Action Expert 的少量锚点速度场保留主要轨迹方向，再训练轻量残差纠偏模型补偿锚点外推相对于原始多步去噪的残差误差。第四章将在此基础上进一步给出速度场相似性分析、压缩 VLM 上下文条件和残差监督目标。

2.3 昇腾平台与 CANN 生态

昇腾平台为国产端侧推理提供了 NPU 硬件和 CANN 软件栈。对于 PyTorch 模型，torch_npu 提供了面向 Ascend NPU 的动态图执行接口，使开发者可以在保留原有 PyTorch 编程范式的基础上迁移模型^[18]。动态图路径适合快速验证模型能否运行、定位算子兼容问题和检查输出正确性，但其运行时调度和 Python 框架开销较大，通常不是端侧低延迟部署的最终形态。

静态图部署通常需要先要将模型导出为 ONNX，再通过 ATC 工具转换为 OM 模型，最后使用 ACL 或 ACLLite 等运行时接口执行推理^[17]。与动态图相比，静态图可以让编译器提前进行图优化、算子选择和精度策略处理，因而更适合稳定部署和延迟优化。但 VLA 模型的结构比常规分类或检测模型更复杂：它们往往包含视觉编码、语言编码、注意力缓存、动作专家、多模态预处理和动态张量形状，导出与编译过程更容易暴露算子不支持、张量形状不固定、精度回退和数据搬运开销等问题。

本文第三章采用“动态图验证 + 静态图优化”的适配路线。动态图阶段用于确

认模型加载、预处理、前向推理和输出检查链路是否正确；静态图阶段进一步处理 ONNX 导出、ATC 编译、OM 推理、混合精度和端到端延迟问题。该分层流程可以避免把“编译成功”误认为“任务正确”，也可以区分数值一致性、任务成功率和推理时延三类验证目标。对于后续去噪蒸馏加速而言，昇腾适配不仅是工程前提，也提供了衡量端侧加速收益的实际执行环境。

2.4 LIBERO 评测基准

LIBERO 是面向语言条件机器人操作任务的仿真评测基准，常用于评估策略模型的知识迁移能力和任务泛化能力^[30]。其任务通常要求模型根据自然语言指令，在桌面环境中完成物体抓取、放置、空间关系调整和多阶段操作。相比只评估单步预测误差，LIBERO 的成功率指标直接反映策略在闭环执行中的任务完成情况，更适合检验 VLA 模型的动作生成质量。

本文主要使用 LIBERO 的四个任务集：spatial、goal、object 和 libero_10。spatial 更强调空间关系理解，goal 更强调目标状态达成，object 关注不同物体交互，libero_10 则包含更复杂的任务组合和长程操作。不同任务集的任务难度和动作阶段差异明显，因此它们不仅用于报告平均成功率，也用于分析去噪蒸馏方法的适用边界。第五章将重点比较原始全量去噪模型、直接减少去噪步数以及残差纠偏方案在这些任务集上的表现。

LIBERO 成功率评测和延迟评测属于不同口径。前者关注闭环任务完成情况，通常受仿真环境、评测回合设置和随机种子影响；后者关注单个动作块的模型推理耗时，需要区分 VLM 前缀、Action Expert 和残差纠偏模型等组件。本文将在实验章节分别报告成功率和延迟，并结合 Ascend 310P、AGX Orin 与 AGX Thor 实测数据分析端侧推理时延，避免混用任务成功率评测平台和速度评测平台。

2.5 本章小结

本章介绍了 VLA 模型类型、生成式动作策略加速、昇腾 CANN 部署链路和 LIBERO 评测基准。VLA 动作表示和推理流程差异要求平台适配覆盖多类型结构；VLM + Action Expert 的多步去噪也为后续压缩提供空间。下一章将给出多类型模型迁移到 Ascend 310P 的动态图、静态图和正确性验证流程。

第三章 VLA 模型昇腾平台适配

3.1 适配目标与整体方案

本章面向多类型 VLA 与具身策略模型，讨论其在 Ascend 310P 上的动态图推理、静态图编译和端到端验证流程。与通用 VLM 或 LLM 相比，当前 VLA 策略的跨任务泛化能力仍更依赖机器人形态、相机配置、动作空间和任务数据分布；在整理桌面、叠衣服等具体操作任务中，通常需要在基础模型之上进行任务或平台相关微调，才能获得可用的闭环控制效果。因此，端侧适配工作的目标不应只是为某一个模型权重转换出固定模型文件，而是提供一套可复用的迁移路径和样例，使不同任务上完成微调的 VLA 模型能够以较低成本迁移到国产算力平台上推理，并进一步接入仿真或真实机器人平台进行验证。

围绕这一目标，本文将昇腾适配拆分为三个层次。第一，保证模型能够正确转换和运行，即在保持模型配置、预处理、动作后处理和关键输出一致的前提下，完成 PyTorch、ONNX 和 OM 之间的迁移。第二，在正确性的基础上进行静态图低延迟优化，通过算子等价改写、混合精度策略、子图拆分和运行时数据搬运优化降低端到端推理时延。第三，提供统一的转换、校验和推理接口，使静态图模型不仅能在离线模拟输入上对齐输出，也能接入 LIBERO、ALOHA 等仿真环境以及后续真实机器人系统，以任务成功率或实际动作行为验证迁移结果。

端侧适配包含三个具体工程难点。第一，动态图保留了 PyTorch 的调试灵活性，但密集小算子、形状整理和索引更新会带来频繁算子下发和 CPU-NPU 往返切换，难以充分利用 NPU 的并行计算能力。第二，静态图部署虽然更接近端侧交付形态，但 ONNX 导出和 ATC 编译过程中可能出现算子不支持、张量形状固化错误和 ONNX/OM 算子形状不一致等问题；由于 VLA 计算图规模较大，完全依赖人工排查会显著增加迁移成本，也不利于将适配经验推广到新的微调模型。第三，模型导出成功并不等价于任务行为正确，少量模拟输入的数值对比无法覆盖真实仿真中的观测分布和动作闭环误差。因此，本文围绕静态图转换、算子形状差异识别和仿真任务级验证建立对应流程，降低迁移新模型时的学习成本和重复调试成本。

整体流程上，本文采用动态图验证与静态图优化相结合的方案。动态图路径基于 torch_npu，将原始 PyTorch 模型迁移到 Ascend NPU 执行，用于快速发现算子兼容和数据类型问题^[18]。静态图路径则将模型导出为 ONNX，并通过 ATC 编译为 OM 模型，

再使用 ACL/ACLLite 运行时进行推理^[17]。动态图路径强调覆盖面和调试效率，静态图路径强调端侧部署形态和推理时延。两条路径共享统一的输入预处理、输出后处理和正确性检查逻辑，从而避免由于接口差异引入额外误差。

在实验组织上，本文首先对多类型具身策略模型进行动态图适配，确认模型加载、输入构造和前向推理链路能够在 Ascend 310P 上执行；随后选择具备部署价值的模型进行静态图转换与混合精度优化，并测量动态图与静态图延迟差异；最后通过数值一致性和任务成功率一致性验证适配后的推理链路是否保持原有行为。该流程为第四章的去噪蒸馏加速提供了工程基础：只有在模型能够正确迁移并具备稳定测量口径的情况下，后续端侧加速比才具有实际意义。

3.2 动态图适配

动态图适配主要依托 torch_npu 将 PyTorch 模型迁移到 Ascend NPU 执行。相较于静态图路径，动态图保留了代码灵活性，适合在迁移早期快速定位问题。本文在动态图阶段重点处理四类内容：模型权重加载、输入预处理、算子执行和输出检查。

模型权重加载阶段需要保证模型配置、权重精度和输入输出维度与原始 GPU 版本一致。对于 VLA 模型而言，输入不仅包括图像和语言指令，还可能包括机器人状态、动作归一化参数、任务文本模板和相机名称等配置。如果这些模型外部配置不一致，即使核心网络能够在 NPU 上执行，也可能产生不同的动作输出。因此，本文将模型配置、预处理参数和推理接口共同纳入适配范围，而不是只迁移单个网络模块。

算子执行阶段主要处理 PyTorch 算子到 Ascend NPU 的映射问题。常规张量计算、矩阵乘和卷积通常可以通过 torch_npu 直接执行，但多模态模型中的动态张量形状、注意力缓存、索引更新和部分后处理算子可能出现不支持或低效执行。对于这类问题，本文优先采用等价算子改写、张量形状固定和局部 CPU 回退执行等方式，使动态图链路先达到可运行状态。与此同时，动态图阶段也会暴露端侧性能瓶颈：当动作生成链路包含大量细粒度算子时，算子下发和 CPU-NPU 同步开销会成为主要时延来源之一。因此，动态图适配的输出不仅包括关键张量形状、数值误差和动作输出检查结果，也为后续静态图转换提供性能动机和接口基准。

正确性验证是动态图适配中不可省略的一步。本文对迁移后的输出与 GPU 参考平台运行结果进行对比，检查关键输出的数值一致性和任务行为一致性。对于连续动作模型，单个张量的局部误差并不总是直接对应任务失败，因此本文同时关注动作向量余弦相似度、动作维度范围和闭环评测结果。在已完成的 500 回合验证中， π_0

在 ALOHA Transfer Cube 任务上的 GPU 参考成功率和 Ascend 310P 成功率均为 91%； $\pi_{0.5}$ 在 LIBERO 4 个任务集上的 GPU 参考平均成功率和 Ascend 310P 平均成功率均为 86%；Diffusion Policy 在 PushT 任务上的 GPU 参考成功率和 Ascend 310P 成功率均为 90%。同时，输出动作向量的余弦相似度高于 99.00%。

3.3 静态图适配与优化

静态图适配以 ONNX 导出、ATC 编译和 OM 推理为核心。相比动态图路径，静态图推理更适合端侧部署和低延迟执行，但也更容易暴露动态张量形状、循环展开、算子转换、混合精度和运行时数据搬运等问题。针对 VLA 模型中常见的 VLM 前缀与 Action Expert 分工，本文将静态图适配拆分为模型拆分、算子兼容性匹配、精度策略和运行时优化四个环节。

首先是模型拆分。许多 VLA 模型在 PyTorch 中将语言图像编码、KV 缓存构造和多步动作去噪写在同一个前向过程中，但 ATC 在转换 ONNX 图时会倾向于展开 Python 侧循环。以连续动作模型的 Action Expert 为例，若直接导出包含 10 步去噪循环的完整图，编译图会复制多份 Action Expert 计算，导致 ONNX/OM 体积显著膨胀，并使导出和 ATC 编译时间明显增加，甚至触发资源分配失败。因此，本文将 VLM 前缀与 Action Expert 的边界显式拆分：VLM 相关部分负责生成语义条件或 KV 缓存，Action Expert 静态图只保留单步去噪，并在运行时由外部循环调用多次。该拆分一方面降低了单个静态图的复杂度，另一方面也为第四章针对 Action Expert 循环的减步和残差纠偏加速提供了明确接口。

其次是算子兼容性匹配。静态图部署不能只依赖 ONNX 导出成功，还需要检查目标后端是否真正支持导出后的算子形态。本文在适配中遇到的典型问题包括拆分算子形式不被 310P 后端支持、时间步嵌入中的 `cos` 节点不被 310P 转换链路支持，以及图优化融合后产生张量形状错误。一个具体案例出现在 π_0 的 RMSNorm 相关路径中：输入张量在 $1 \times 304 \times 2048$ 维度上先平方再对最后一维取均值，ONNX 图中期望输出为 $1 \times 304 \times 1$ ，但 ATC/OM 转换后的融合节点输出被错误固化为 $1 \times 1 \times 1$ ，导致后续归一化结果偏离。针对该问题，本文将 `mean` 显式改写为 `sum / count`，绕开错误融合路径；对于 `cos` 节点，则采用等价的 $\sin(x + \pi/2)$ 改写；对于不稳定的掩码构造和索引更新路径，则将 `where`、`ScatterND` 等节点移出或替换为后端更易支持的写法。

再次是精度策略。对于矩阵乘、卷积等主要计算，静态图通常可以使用 `fp16` 或

混合精度获得较好吞吐；但归一化、Rotary 位置编码、注意力归一化和加性注意力掩码等路径对数值范围更敏感，若全部强制为 fp16，可能在极大负掩码或归一化前后产生 NaN 或明显误差。以 π_0 的 LIBERO 模型权重为例，导出时保留模型主体的 fp16 计算，但将 RMSNorm、Rotary 位置编码、注意力归一化和加性注意力掩码等小规模关键路径置于 fp32，并把注意力掩码的构造移出 ONNX 图作为显式输入。该过程表明，静态图优化并不是简单调用导出命令，而需要围绕目标硬件的算子覆盖、数值稳定性和图结构可编译性共同调整。

为了减少手工排查成本，本文还建立了 ONNX/OM 不一致自动定位脚本框架。该框架不只检查模型是否能够编译成功，而是围绕输入输出接口、图结构扫描、组件内部接口、端到端正确性和仿真平台测试五个层次生成检查结果。脚本首先导出 ONNX 模型的输入输出名称、数据类型和张量形状，并与 OM 运行时输入输出清单对齐，用于发现动态张量形状固化错误、掩码维度错误和子图拆分边界不一致。随后，脚本比较 ONNX 与 OM 转换前后的关键节点、张量形状推断结果和 ATC 日志，将算子不支持、张量形状异常变化和 ATC 警告整理为候选问题点。对于由多个静态图组件串联的模型，脚本进一步检查组件之间的 KV 缓存、前缀掩码、动作状态等内部接口，确认其数据类型、大小和维度一致。最后，脚本用同一组固定输入分别运行 PyTorch、ONNX Runtime 和 OM，计算完整动作块的余弦相似度、最大绝对误差和平均绝对误差；在条件允许时，还将 OM 静态图接入仿真评测流程，用任务成功率验证静态图行为是否保持一致。

表 3-1 静态图不一致自动定位框架的主要检查项

检查层次	自动提取内容	主要定位问题
输入输出接口	ONNX/OM 输入输出名称、数据类型、张量形状	动态张量形状固化错误、掩码维度不一致、模型入口或出口接口错误
图结构扫描	ONNX/OM 关键节点、张量形状推断结果、ATC 警告节点	算子不支持、融合后张量形状错误、后端低效路径
组件内部接口对齐	多部分静态图之间的 KV 缓存、前缀掩码、动作状态和单步输出	组件之间数据类型、大小或维度不一致
端到端正确性测试	固定输入下的完整动作块、余弦相似度、最大/平均误差	检查静态图输出与参考实现是否一致，以及多步误差是否累积
仿真平台测试	LIBERO、ALOHA 等任务的多用例成功率	检查离线数值一致性之外的任务行为一致性

在 π_0 静态图适配中，上述脚本框架定位并验证了多个具体问题。早期 fp16 加性

注意力掩码会在 PyTorch fp16 端到端回放中产生 NaN，因此最终改为 fp32 掩码输入；ONNX 导出图中的掩码构造若保留在图内，会引入后端不友好的 Where 节点，因此改为由预处理阶段生成加性掩码；时间步嵌入中的 Cos 节点由于 310P 转换链路不支持而通过等价三角函数改写消除；RMSNorm 中 $1 \times 304 \times 2048$ 张量的平方取均值路径则通过 $\text{sum} / \text{count}$ 手动替换，避免 ATC 融合后将 $1 \times 304 \times 1$ 错误变为 $1 \times 1 \times 1$ 。修复后， π_0 分段 ONNX 图中 Cos、Where 和 ScatterND 的数量均为 0；ONNX 与 OM 的组件级校验中，VLM 前缀子图的 KV 缓存余弦相似度和 Action Expert 单步输出余弦相似度分别高于 99.999%；固定输入的分段端到端校验中，最终动作与 ONNX 基线的最大绝对误差为 0.001464844，平均绝对误差为 0.000239642。这些结果表明，自动化检查脚本能够把“转换失败或输出异常”进一步分解为具体的算子、张量形状、掩码或子图边界问题，使静态图适配从一次性试错转为可复查的定位流程。

最后，运行时优化需要关注模型执行之外的数据搬运和主机侧调度开销。VLA 推理通常包含多次子图调用和多模态张量输入，若每次调用都重复传入大块 KV 缓存或中间状态，端到端延迟会明显高于单个静态图的纯 NPU 计算时间。以 SmolVLA 在 Ascend 310P 上的静态图部署为例，仅 NPU 口径下，VLM/part1 的计算时间为 63.524 ms，Action Expert/part2 单步计算时间为 6.748 ms；若按原始 10 步去噪估算，纯 NPU 计算约为 $63.524 + 10 \times 6.748 = 131.004$ ms。而在 ACLLite 主机侧串联 OM 的端到端口径下，原始动作块延迟为 147.290 ms，比纯 NPU 估算高约 16.286 ms，约占端到端延迟的 11.1%，说明子图串联、输入输出拷贝和 Python/ACL 调用调度会带来不可忽略的额外开销。因此，本文在 SmolVLA 的静态图链路中将动作块推理拆分为 VLM/part1、Action Expert/part2 和残差纠偏模型等组件，并分别测量仅 NPU 与主机侧串联口径。该拆分帮助我们理解后续的优化方向。

3.4 适配结果与验证

本文在 Ascend 310P 上对多类型具身策略模型进行了动态图和静态图适配。表 3-2 给出了六组适配案例在动态图与静态图口径下的推理延迟对比，用于说明动态图到静态图部署的整体收益。表中的静态图结果包含 ONNX/ATC/OM 转换、混合精度和算子兼容性修复后的部署口径；SmolVLA 使用分段 OM 主机侧串联的动作块口径，即 VLM 第 1 部分与 10 次 Action Expert 第 2 部分串联。

表 3-2 显示，六组模型的平均延迟由 1049.80 ms 降至 360.15 ms，整体加速比约为 2.91 $\times$ ，表明 ONNX/ATC/OM 静态图部署和混合精度优化能够降低多数 VLA 与具身

表 3-2 Ascend 310P 动态图与静态图推理延迟对比

模型	计时口径	动态图延迟/ms	静态图延迟/ms	静态/动态	加速比
ACT	动作分块策略	14.26	2.50	17.5%	5.70×
Diffusion Policy	PushT 动作生成	1853.00	500.80	27.0%	3.70×
SmolVLA	分段动作块	488.34	147.29	30.2%	3.32×
π_0	分段动作块	818.00	350.30	42.8%	2.34×
OpenVLA	动作生成	2280.00	810.00	35.5%	2.81×
$\pi_{0.5}$	分段动作块	845.20	350.00	41.4%	2.41×
平均	六组均值	1049.80	360.15	34.3%	2.91×

策略模型的推理时延。其中 ACT 的结构较轻,静态图延迟降至 2.50 ms; Diffusion Policy 和 OpenVLA 的原始动态图延迟较高,静态图优化后也获得了较大收益; SmolVLA 的静态图动作块延迟为 147.29 ms, 用于说明该微调权重版本下的静态图部署收益。

除延迟外,本文还进行了正确性验证。静态图转换的一般做法是先使用模拟输入比较 PyTorch、ONNX 或 OM 输出,但该方式只能覆盖有限输入分布,难以发现真实闭环任务中由观测变化触发的不一致。因此,本文进一步将 OM 静态图接入 LIBERO、ALOHA 等基于 MuJoCo 的仿真任务评测流程,通过多任务、多用例自动运行比较成功率。表 3-3 汇总了已完成任务级验证的代表性结果。

表 3-3 Ascend 310P 任务级正确性验证结果

模型/策略	任务或基准	评测回合数	GPU 参考成功率	Ascend 310P 成功率	数值一致性	说明
π_0	ALOHA Transfer Cube	500	91%	91%	动作向量余弦 > 99.00%	双平台任务成功率一致
$\pi_{0.5}$	LIBERO 4 个任务集	500	86%	86%	动作向量余弦 > 99.00%	4 个任务集平均成功率一致
Diffusion Policy	PushT	500	90%	90%	动作向量余弦 > 99.00%	双平台任务成功率一致

表中余弦相似度对应模型最终输出的动作向量,而不是中间隐藏状态或单个调试张量。表中结果显示, π_0 在 ALOHA Transfer Cube 任务上达到 91% 成功率, $\pi_{0.5}$ 在 LIBERO 4 个任务集上达到 86% 平均成功率, Diffusion Policy 在 PushT 任务上达到 90% 成功率,且三组结果在 GPU 参考平台和 Ascend 310P 上一致;数值一致性验证中动作向量余弦相似度高于 99.00%。这表明适配后的推理链路不仅能够降低时延,也能够保持任务行为正确。对于后续推理加速实验,先完成正确性验证具有直接意义:若平台迁移本身已经改变了动作分布,则很难判断残差纠偏方法的真实收益。通过先完成正确性验证,再进行端侧延迟测量,本文将平台适配误差和方法加速收益进行了区分。

3.5 本章小结

本章围绕 VLA 模型的昇腾平台适配, 介绍了动态图验证、静态图转换、混合精度优化和正确性验证流程。实验结果表明, 多数模型在 Ascend 310P 静态图路径下获得延迟收益, 六组模型平均延迟从 1049.80 ms 降至 360.15 ms; 同时, π_0 在 ALOHA Transfer Cube 任务上达到 91% 成功率, $\pi_{0.5}$ 在 LIBERO 4 个任务集上达到 86% 平均成功率, Diffusion Policy 在 PushT 任务上达到 90% 成功率, 且 GPU 参考平台与 Ascend 310P 结果一致, 动作向量余弦相似度高于 99.00%。这些结果表明, 主流 VLA 与具身策略模型可以在国产端侧平台上形成可复用的部署路径。下一章将在该平台基础上, 进一步讨论如何针对 VLM + Action Expert 连续动作生成模型降低 Action Expert 多步去噪开销。

第四章 锚点残差去噪推理加速方法

4.1 VLA 推理时间分布

第三章已经说明,多类型 VLA 与具身策略模型可以通过动态图验证、静态图转换和任务级正确性校验迁移到 Ascend 310P 上。本章进一步讨论推理加速问题。本文关注的对象是视觉语言模型 (VLM) + 动作专家模块 (Action Expert, AE) 类连续动作生成模型: VLM 负责根据视觉观测、语言指令和机器人状态形成语义条件, Action Expert 在该条件下通过多步去噪或速度场积分生成一个动作块。该结构的特点是, VLM 主干参数量通常更大,但每个动作块中只执行一次; Action Expert 参数量相对较小,却需要在同一个动作块内重复执行多次。因此,端到端延迟并不只由参数量决定,还与模型调用次数和中间上下文搬运开销密切相关。

图 4-1 对比了原始 VLA 推理架构与本文的锚点残差去噪优化架构。左侧原始流程中,视觉图像、机器人状态和语言指令分别编码后进入 VLM 主干模型, VLM 生成的条件信息通过 KV 缓存传递给 Action Expert; Action Expert 从随机动作空间出发,多次执行全量去噪模型,最终输出连续动作。右侧优化流程保留相同的 VLM 条件构造路径,但在动作生成阶段只保留少量全量去噪锚点,并将裁剪后的 KV 缓存和锚点状态输入轻量纠偏模型,使其补偿少步外推与全量去噪轨迹之间的残差。因此,本章的加速目标是在保证端到端任务成功率尽可能接近原始模型的前提下,减少去噪模型的执行次数并提高推理速度。

本文选择 SmolVLA、 π_0 和 $\pi_{0.5}$ 作为主要优化对象,是因为三者都属于当前有代表性的 VLM + Action Expert 连续动作模型,并覆盖从轻量级到数十亿参数级的不同模型规模。 π_0 是基于 PaliGemma VLM 和 Flow Matching Action Expert 的通用机器人控制模型,其论文将 Action Expert 设计为约 300M 参数的较小专家模块,用于多步连续动作生成^[5]。 $\pi_{0.5}$ 在 π_0 基础上加入更强的开放世界泛化训练和高层语义预测,模型技术细节中继续沿用 VLM + Action Expert 的双分支结构^[6]。SmolVLA 则面向低成本训练和部署,论文报告主模型约 0.45B 参数、Action Expert 约 100M 参数,并在 LIBERO 与 Meta-World 等仿真基准上达到与更大 VLA 相当的表现^[7];本文用于 LIBERO 成功率和 ARD 训练的 SmolVLA 微调模型权重实测约 0.605B 参数,其中 VLM 侧约 507M、Action Expert 约 97M。同时,LIBERO-PRO 将 OpenVLA、 π_0 和 $\pi_{0.5}$ 作为先进 VLA 代表模型进行鲁棒性评测,说明这些模型已经成为 LIBERO 系列评测中的重要参照^[31]。

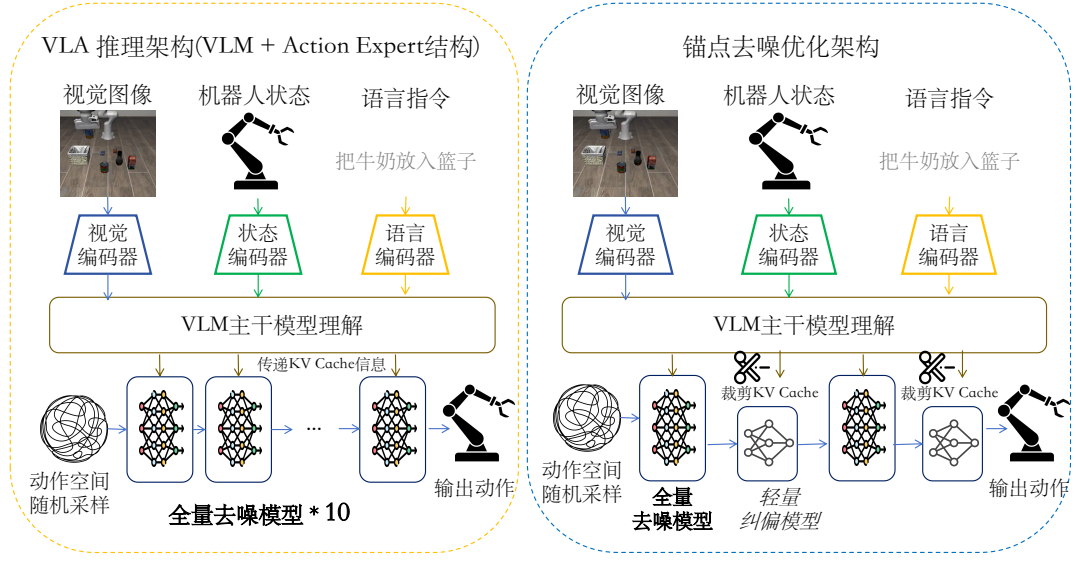


图 4-1 原始 VLA 推理结构与锚点残差去噪优化结构对比。左侧为 VLM + Action Expert 的常规推理路径：VLM 主干根据视觉、状态和语言条件构造上下文，Action Expert 在动作空间中重复执行全量去噪模型。右侧为本文的优化路径：VLM 条件构造保持不变，动作生成阶段通过少量全量去噪锚点和轻量纠偏模型替代多数 Action Expert 调用，并使用裁剪后的 KV 缓存提供任务上下文。

因此，本文覆盖约 0.6B 级轻量模型和 3B 以上 VLA 模型，并包含面向开放世界泛化增强的 $\pi_{0.5}$ 变体，适合用来分析不同规模和训练目标下 Action Expert 多步循环的推理瓶颈和加速收益。

以 SmolVLA、 π_0 和 $\pi_{0.5}$ 为例，这类连续动作模型的常规推理过程通常使用 10 步 Action Expert 迭代生成长度为 50 的动作块。为了分析加速方法应优先作用于哪一部分，表 4-1 给出不同硬件上的动作块延迟拆分。表中 Action Expert 分项由 denoise_step 钩子直接累计，对应 10 次 Action Expert 调用；VLM 前缀包含图像、语言和状态条件编码及前缀 KV 缓存构造；其他开销包括 Python 调度、数据搬运、同步和计时残差等。表中结果均按 PyTorch 组件级延迟测试口径整理，其中 Ascend 310P 通过 torch_npu 后端执行。

图 4-2 将表 4-1 中的时间拆分以图形方式呈现。SmolVLA 在三个端侧平台上的 Action Expert 占比均较高，而 π_0 与 $\pi_{0.5}$ 在 Ascend 310P 上的 VLM 前缀占比更高。这一差异说明，ARD 的端到端加速上限会随模型结构和平台运行时开销而变化，但 Action Expert 多步循环仍是各模型中稳定存在的可压缩部分。

表 4-1 VLA 动作块推理时间拆分

模型	平台	总/ms	VLM/ms	VLM%	AE/ms	AE%	其他/ms	其他%
SmolVLA	Ascend 310P	377.37	121.63	32.2%	246.39	65.3%	9.35	2.5%
SmolVLA	AGX Orin	1399.97	345.60	24.7%	1038.95	74.2%	15.42	1.1%
SmolVLA	AGX Thor	307.63	44.60	14.5%	261.79	85.1%	1.24	0.4%
π_0	Ascend 310P	467.50	296.14	63.3%	161.62	34.6%	9.74	2.1%
π_0	AGX Orin	832.13	244.78	29.4%	480.01	57.7%	107.34	12.9%
π_0	AGX Thor	251.32	80.96	32.2%	128.99	51.3%	41.37	16.5%
$\pi_{0.5}$	Ascend 310P	606.48	365.31	60.2%	231.86	38.2%	9.31	1.5%
$\pi_{0.5}$	AGX Orin	1017.16	304.89	30.0%	600.18	59.0%	112.09	11.0%
$\pi_{0.5}$	AGX Thor	277.37	95.76	34.5%	140.98	50.8%	40.63	14.7%

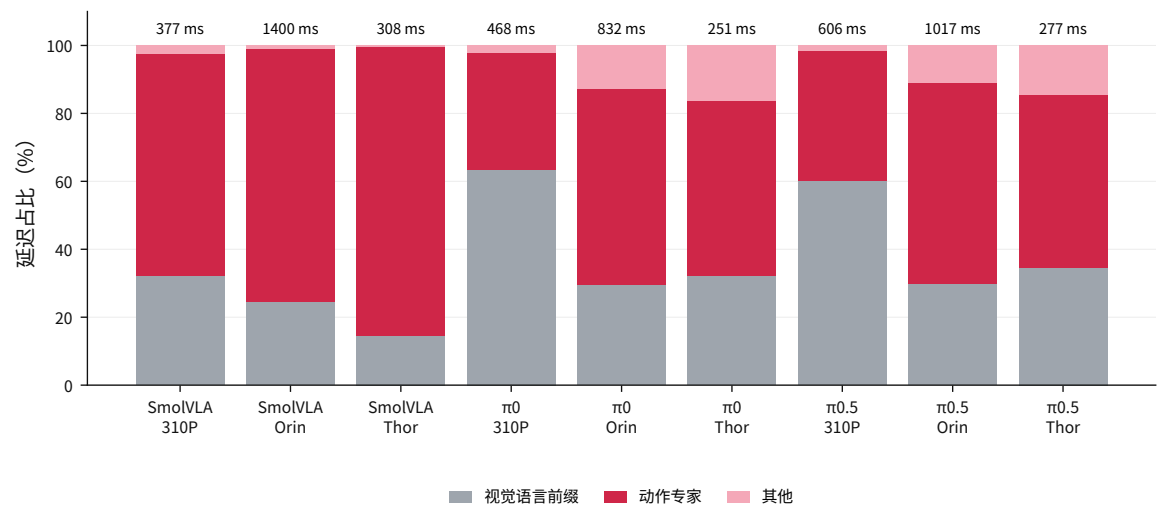


图 4-2 VLA 动作块推理时间组成。柱内颜色表示 VLM 前缀、Action Expert 和其他运行时开销在总延迟中的占比，柱顶标注对应平台上的总延迟。该图用于展示各模型在不同硬件上的瓶颈分布，与表 4-1 使用相同数据。

同一 VLA 架构在不同模型权重、相机数量和状态/动作维度下会产生不同输入输出张量形状,进而影响绝对延迟。SmolVLA 的 Ascend 310P 适配实验使用公开基础权重; LIBERO 成功率验证和纠偏模型训练使用公开 LIBERO 微调权重。因此,表 4-1 主要用于比较 VLM 前缀、Action Expert 与其他运行时开销的相对分布,而不将不同权重的绝对延迟直接等同。SmolVLA 的 AGX Orin 行采用完整 VLM 前缀复测结果,计时范围与表中 VLM 前缀、Action Expert 和其他开销的拆分口径保持一致。

这些拆分结果给出两点直接动机。第一, Action Expert 多步循环是跨平台可压缩对象。SmolVLA 在 AGX Orin 和 AGX Thor 上的 Action Expert 占总延迟的 74.2% 至 85.1%,在 Ascend 310P 上为 65.3%; π_0 和 $\pi_{0.5}$ 的 VLM 前缀占比更高,但 Action Expert 仍占据约 34.6% 至 59.0% 的端到端延迟。第二,总延迟并不是 VLM 前缀与 Action Expert 的简单相加,其他部分还包含前后处理与运行时固定开销;当 Action Expert 调用次数减少后,这些固定开销会迅速成为端到端加速上限。因此,将本文的加速目标聚焦在 Action Expert 多步循环上是合理的:该部分具有重复调用特征,减少调用次数可以直接降低计算开销,并有望取得可观的端到端收益。同时,VLM 前缀压缩同样具有潜在价值,本文的选择只是优先处理由去噪步数控制的重复计算部分,并要求纠偏与调度开销控制在节省的去噪时间之内。

4.2 问题形式化

设一个动作块的内部动作状态为 $x_i \in \mathbb{R}^{H \times D}$,其中 H 表示动作块长度, D 表示内部动作维度。在本文使用的 SmolVLA LIBERO 权重中, $H = 50$, $D = 32$;真实执行时只使用其中与机器人控制对应的动作维度。设 VLM 前缀产生的条件信息为 c , Action Expert 的速度场预测函数为 $v_\theta(x_i, t_i, c)$ 。原始全量去噪模型以 $N = 10$ 步离散积分生成动作块,可写为

$$x_{i+1} = x_i + \Delta t \cdot v_\theta(x_i, t_i, c), \quad i = 0, \dots, N-1, \quad (4-1)$$

其中 $\Delta t = -1/N$ 表示当前实现中的时间步长符号约定。全量去噪输出 x_N 作为教师轨迹的终点,具有较高的任务成功率,但需要执行 N 次 Action Expert。

直接减步可以将 N 次调用减少到 M 次,但会改变轨迹离散化过程。本文采用不同的思路:在少量关键时间步执行原始 Action Expert,得到锚点状态和锚点速度,再由轻量残差纠偏模型预测线性外推与全量去噪轨迹之间的偏差。对于从第 s 个状态

开始、覆盖 K 个离散步的纠偏块，先执行一次锚点去噪得到

$$v_s = v_\theta(x_s, t_s, c). \quad (4-2)$$

若只使用锚点速度进行线性外推，则目标状态近似为

$$\tilde{x}_{s+K} = x_s + K\Delta t \cdot v_s. \quad (4-3)$$

全量去噪轨迹中同一位置的教师状态记为 x_{s+K}^T ，则残差监督目标为

$$r_{s,K}^T = x_{s+K}^T - \tilde{x}_{s+K}. \quad (4-4)$$

残差纠偏模型 Δ_ϕ 的输入包括锚点状态 x_s 、锚点速度 v_s 、时间 t_s 以及压缩后的上下文条件 $\bar{c}$ ，输出为

$$\hat{r}_{s,K} = \Delta_\phi(x_s, v_s, t_s, \bar{c}). \quad (4-5)$$

最终估计的目标状态为

$$\hat{x}_{s+K} = x_s + K\Delta t \cdot v_s + \hat{r}_{s,K}. \quad (4-6)$$

因此，本文方法并不让轻量模型重新生成完整动作块，也不替代 VLM 或 Action Expert 的语义理解能力，而是让其学习“少步锚点线性外推相对于全量去噪轨迹的残差”。这一形式将学习目标限制在较小的纠偏空间中，同时保留了原始 Action Expert 的速度场先验。

4.3 直接减步数的局限性

最直接的加速方式是减少 Action Expert 的迭代步数。例如，将 10 步去噪改为 N 步 ($N < 10$)，可以降低模型调用次数；在只看推理时间时，这通常能得到较高加速比。但是，机器人策略的质量不能只由单步张量误差衡量，必须通过闭环任务成功率验证。动作轨迹中的局部偏差可能在抓取、接触和放置阶段被放大，最终导致任务失败。

表 4-2 汇总了 SmolVLA 在 LIBERO 代表任务上的步数消融结果，并补充 AGX Thor 上同一微调权重的 PyTorch 动态图动作块延迟。该表使用每个任务集中部分代表任务的均值，不是完整 4 个任务集全量任务均值，其作用是说明步数压缩的风险边界。不同任务集对去噪步数的敏感性差异明显：libero_goal 在 step1 时代表任务均值为 0%，step2 也只有 55%；而 libero_spatial 和 libero_object 对 stepN

表 4-2 SmolVLA 代表任务直接减步数成功率与 AGX Thor 延迟

任务集	step1	step2	step4	step6	step8	step10
libero_goal	0%	55%	75%	70%	85%	70%
libero_spatial	75%	75%	70%	70%	85%	90%
libero_object	75%	80%	80%	90%	85%	95%
libero_10	25%	35%	25%	45%	35%	35%
平均	43.8%	61.3%	62.5%	68.8%	72.5%	72.5%
Thor 延迟/ms	72.91	99.21	151.20	202.71	253.96	306.74
Thor 加速	4.21×	3.09×	2.03×	1.51×	1.21×	1.00×

直接减步更鲁棒。Thor 延迟结果显示，step1 可以达到较高加速比，但若为了保持较高成功率而保留 step6 或 step8，端到端加速空间会明显受限。

直接减步实验说明，Action Expert 轨迹确实存在压缩空间，但“只执行少数步并直接输出”的策略不够稳定。对于较简单或短程的任务，少步推理可能已经接近原始模型；对于需要精细接触、多阶段状态转换或较强任务泛化能力的任务，少步推理容易偏离全量去噪轨迹。因此，本文需要一种能够充分利用全量去噪模型轨迹质量、同时降低推理开销的方法：既利用少步锚点减少 Action Expert 调用次数，又通过轻量纠偏模型补偿少步外推带来的累计误差。

4.4 锚点残差去噪方案设计

4.4.1 设计观察：速度场相似性与累计偏移

锚点残差去噪的核心假设是：相邻去噪步的速度场方向具有较高一致性，因此少量锚点速度可以提供后续轨迹的大致方向；但多步累积后仍会产生偏移，因此仅做线性外推不足以替代全量去噪。本文用 SmolVLA 与 π_0 的速度场数据验证这一点。表 4-3 给出了相邻速度余弦相似度的最小值、第 10 步线性外推的余弦偏差，以及相邻速度场模长比范围。其中余弦偏差定义为 $1 - \cos(10v_0, \sum_{i=0}^9 v_i)$ ，用于衡量初始锚点速度线性外推方向与全量累计速度方向之间的差异；相邻模长比定义为 $\|v_i\|_2 / \|v_{i-1}\|_2$ ，用于判断相邻速度场幅值是否接近。

图 4-3 给出了同一组速度场统计的可视化结果。图中 a、b、c 分别对应相邻速度方向一致性、第 10 步线性外推方向偏差和相邻速度模长比范围；其中 L10 对应 libero_10 任务集。可视化结果与表 4-3 一致：相邻速度方向高度一致，但长跨度

表 4-3 速度场相邻相似度、线性外推偏差与模长比概览

数据集与模型	相邻余弦相似度最小值	第 10 步余弦偏差	相邻模长比范围
SmolVLA spatial	0.9874	0.0378	0.980–1.000
SmolVLA libero_10	0.9889	0.0240	0.981–0.999
π_0 spatial	0.9699	0.0917	0.961–1.028
π_0 libero_10	0.9744	0.0455	0.978–1.009

线性外推仍存在偏差，因此需要残差纠偏模型补偿累计误差。

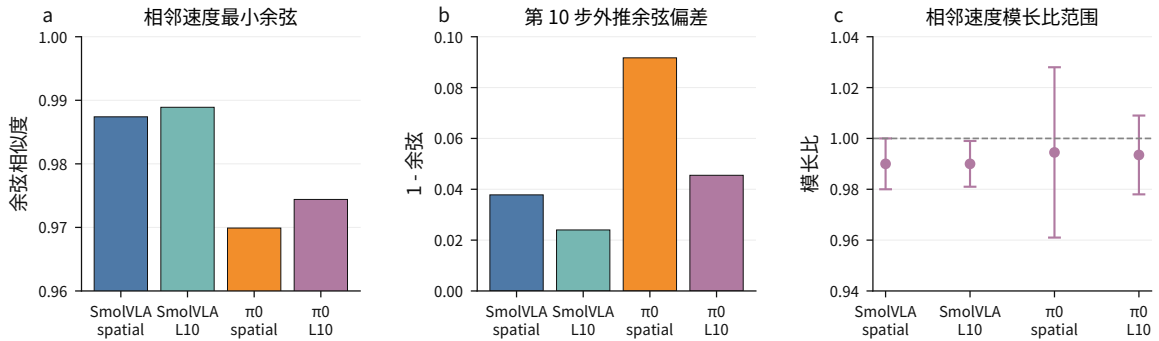


图 4-3 速度场相邻一致性与线性外推偏差。a 为相邻速度余弦相似度最小值，b 为第 10 步线性外推的余弦偏差，c 为相邻速度场模长比范围。图中 L10 对应 libero_10 任务集。

表中结果表明，SmolVLA 在两个任务集合上的相邻余弦相似度均不低于 0.987， π_0 也不低于 0.970，说明相邻速度方向高度一致。相邻模长比整体接近 1，SmolVLA 的范围约为 0.980 至 1.000， π_0 的范围约为 0.961 至 1.028，说明相邻速度场的幅值也基本稳定，但 π_0 在 spatial 任务上波动略大。与此同时，第 10 步线性外推的余弦偏差达到 0.0240 至 0.0917，说明第一步速度可以提供强方向先验，但直接外推完整轨迹仍存在可观方向偏差。

该观察直接支撑本文的残差化设计。若速度场相邻方向完全不一致，单个锚点速度无法提供有效先验，轻量纠偏模型需要重新学习完整 Action Expert 行为；若线性外推已经完全准确，则不需要额外纠偏。实际结果处于二者之间：锚点速度足以给出轨迹主方向，但累计偏差需要被建模。因此，残差纠偏模型只需学习全量去噪轨迹相对于锚点线性外推的偏差，而不是从零预测最终动作状态。

4.4.2 整体方法

从推理过程看, 原始 SmolVLA 首先由 VLM 前缀对图像、语言指令和机器人状态进行编码, 并得到可供后续 Action Expert 访问的 KV 缓存。随后模型从噪声动作块 x_0 出发, 在 $N = 10$ 个时间步上反复调用同一个 Action Expert: 第 i 次调用根据当前状态 x_i 、时间 t_i 和 VLM 条件上下文预测速度 v_i , 再按式 4-1 更新到 x_{i+1} 。因此, VLM 前缀在一个动作块中只计算一次, 而 Action Expert 的前向计算会随去噪步数重复执行。本文数学记号采用从 0 开始的下标; 实验配置中的 step1 表示执行 1 次 Action Expert 调用, 不表示速度或状态下标从 1 开始。

本文提出锚点残差去噪 (Anchor-Residual Denoising, ARD) 方法, 将上述循环改写为“少量 Action Expert 锚点调用 + 轻量残差纠偏”的推理结构。以单锚点残差纠偏为例, 模型仍先执行 VLM 前缀并保留前缀 KV 缓存; 随后只在初始状态 x_0 上调用一次 Action Expert, 得到锚点速度 v_0 。若直接用 v_0 外推到最终步, 可得到 $x_0 + N\Delta t \cdot v_0$; ARD 在此基础上引入残差纠偏模型, 输入锚点状态、锚点速度、时间位置以及 VLM 条件上下文, 预测线性外推相对于全量 10 步教师轨迹的残差, 最终得到纠偏后的动作块。这样, 原始 10 次 Action Expert 调用被替换为 1 次 Action Expert 调用和 1 次轻量纠偏前向。

在更稳健的两阶段配置中, ARD 将长跨度纠偏拆成两个较短区间。当前主要使用中间步为 $m = 5$ 的配置: 第一阶段在 x_0 处执行初始锚点去噪, 利用第一阶段纠偏从 x_0 估计到中间状态 $\hat{x}_5$; 随后在 $\hat{x}_5$ 处再次调用 Action Expert 得到新的锚点速度 v_5 , 并由第二阶段纠偏从 $\hat{x}_5$ 估计最终状态 $\hat{x}_{10}$ 。这种流程用 2 次 Action Expert 调用和 2 次纠偏前向替代原始 10 次 Action Expert 调用。

残差纠偏模型的设计围绕两个约束展开。第一, 纠偏不能只依赖动作状态本身, 还需要感知语言指令、视觉观测和机器人状态形成的任务上下文; 因此模型复用 VLM 前缀 KV 缓存, 并通过层混合、KV 投影和可学习查询交叉注意力将完整上下文压缩为低维条件表示, 避免直接把全部 KV 展开后输入 MLP。第二, 训练目标需要更贴近闭环控制效果。除基础残差 MSE 外, 本文在训练中根据回合是否成功对样本损失加权, 使模型更多拟合成功执行轨迹中的有效残差分布。

与直接训练学生模型替代 Action Expert 相比, ARD 的学习目标更受约束。学生模型若直接预测最终动作块, 需要同时学习语义条件理解、连续控制分布和多步轨迹积分效果; ARD 则将原始 Action Expert 的锚点速度作为输入, 使轻量模型只关注少步外推产生的偏差。与 LLM 投机解码或 ASDSV 式投机验证不同^[14-16], ARD 不包含

离散词元的候选生成、验证和接受/拒绝流程，也不要求目标模型对每个候选动作进行验证。它更接近一种面向连续动作轨迹的去噪蒸馏：以全量去噪轨迹为教师信号，以锚点线性外推为强先验，以残差为监督目标。

4.4.3 残差纠偏模型结构

本文以 SmolVLA 为主要实现对象说明残差纠偏模型结构。模型输入分为状态分支和上下文分支。状态分支包含锚点状态 $x_s \in \mathbb{R}^{50 \times 32}$ 、锚点速度 $v_s \in \mathbb{R}^{50 \times 32}$ 以及时间标量 t_s ，三者展平后拼接为 $50 \times 32 \times 2 + 1 = 3201$ 维向量，再通过两层 MLP 得到状态表示。上下文分支接收 VLM 前缀 KV 缓存和前缀掩码。对于 SmolVLA，KV 中每层键/值展平后拼接形成上下文特征；模型通过可学习的层混合将 32 层 KV 软压缩到较少的压缩层，再映射到固定维度的上下文词元。

压缩后的上下文词元由若干可学习查询通过交叉注意力读取。每个查询首先与压缩 KV 词元做注意力交互，随后经过 FFN 更新；在约 9.237M 参数的主线纠偏模型中，模型额外堆叠多个交叉注意力模块，以增强对压缩上下文的读取能力。最后对查询表示做平均池化得到全局上下文向量 h_c 。状态向量 h_s 与上下文向量 h_c 拼接后输入融合 MLP，输出与动作块同形状的归一化残差 $\hat{f}_{s,K}$ 。

表 4-4 总结了残差纠偏模型的主要模块。早期 SmolVLA 纠偏模型使用 8 个压缩层、8 个可学习查询和 1 个交叉注意力模块，参数量约 5.286M；后续主线纠偏模型将压缩层和可学习查询均扩大到 16 个，并将交叉注意力模块增加到 6 个，参数量约 9.237M。相对于 SmolVLA 的 Action Expert 约 97M 参数和 VLM 约 507M 参数，该纠偏模型仍保持轻量。

不同 VLA 模型上的残差纠偏模型并不完全同构同规模。 $\pi_{0.5}$ 的单锚点残差纠偏模型使用 18 层前缀输入、8 个压缩层和 8 个可学习查询，参数量约 5.253M，与早期 SmolVLA 纠偏模型规模接近，但由于 VLM 层数和 KV 特征维度不同，二者并不是完全相同的结构。本文选择扩大 SmolVLA 纠偏模型，是因为早期约 5.286M 参数的纠偏模型在更复杂的闭环任务上表达能力不足：例如 libero_goal 代表任务评测中，step1 为 20.0%，约 5.286M 参数的单锚点纠偏模型为 42.0%，而约 9.237M 参数的单锚点纠偏模型恢复到 64.0%。扩大上下文压缩容量和交叉注意力深度后，纠偏模型能够更充分利用 VLM 前缀条件，因此被作为本文主线配置。

表 4-4 残差纠偏模型的主要模块

模块	输入	作用
状态分支	x_s, v_s, t_s	编码当前动作噪声状态、锚点速度和时间位置
KV 压缩	VLM 前缀 KV 缓存	将完整上下文压缩为较少的上下文词元，降低输入规模
可学习查询交叉注意力	可学习查询与压缩 KV 词元	从视觉语言上下文中提取与纠偏相关的全局条件
融合 MLP	状态表示与上下文表示	输出归一化残差，再反归一化并加回线性外推结果

4.4.4 训练数据构造与目标函数

训练数据来自全量去噪教师模型的轨迹采样记录。对于每个样本，系统记录全量去噪过程中的状态轨迹 $x_0, \dots, x_N$ 、速度轨迹 $v_0, \dots, v_{N-1}$ 、时间步、VLM 前缀 KV 缓存、前缀掩码以及任务和回合元信息。根据给定的区间配置，从轨迹中选择锚点状态 x_s 、锚点速度 v_s 和目标教师状态 x_{s+K}^T ，按式 4-4 构造残差目标。表 4-5 给出本文主线 SmolVLA ARD 训练数据覆盖范围。

表 4-5 SmolVLA ARD 训练数据覆盖范围

数据范围	任务覆盖	教师样本数	训练/验证划分	训练配置关系
LIBERO 4 个任务集	每个任务集的任务 0-3	每个任务 2×10 个样本	3 : 1	未加权与加权配置使用相同数据

表中的 2×10 指每个任务采集 2 个批次、每个批次包含 10 个并行教师回合采样。每个回合会进一步展开为多条包含完整去噪轨迹和任务元信息的教师轨迹采样记录，而不是单个去噪时间步。在最终加权双锚点配置中，离线缓存约包含 4.2×10^4 条训练轨迹采样记录；由于双锚点方案将每条记录拆分为前后两个纠偏区间，对应约 8.4×10^4 条区块级训练样本。验证集约包含 1.7×10^4 条轨迹采样记录。未加权与加权训练使用完全相同的教师轨迹采样记录和训练/验证划分，二者只在损失函数中的回合级样本权重上不同。因此，后续比较未加权与加权配置时，差异主要来自损失权重设计，而不是训练数据覆盖范围改变。

在已有教师轨迹缓存的基础上，最终加权双锚点残差纠偏模型约含 9.2M 个参数，可在单张 NVIDIA H20 GPU 上约 20 分钟完成训练。该训练成本远低于重新微调整个 VLA 主模型或训练完整学生去噪模型，也体现了本文方法面向具体任务快速构建纠

偏模型的特点。

训练时先对状态输入和残差目标分别统计均值与标准差，并在训练集中使用归一化后的 x_s 、 v_s 、 t_s 和 $r_{s,K}^T$ 。基础损失采用 MSE：

$$\mathcal{L}_{res} = \left\| \Delta_\phi(x_s, v_s, t_s, \bar{c}) - r_{s,K}^T \right\|_2^2. \quad (4-7)$$

该目标直接约束纠偏模型逼近全量去噪轨迹相对于锚点线性外推的误差项。推理时，模型输出归一化残差，运行时使用训练阶段保存的统计量反归一化，再根据式 4-6 得到纠偏后的动作状态。

考虑到平均 MSE 与闭环任务成功率之间并不完全一致，本文在训练中还引入了回合级加权损失配置。其基本形式是在残差 MSE 上根据教师轨迹采样记录的执行结果分配样本权重：

$$\mathcal{L}_w = \frac{1}{B} \sum_{b=1}^B \alpha_b \left\| \hat{r}_{s,K}^{(b)} - r_{s,K}^{T,(b)} \right\|_2^2. \quad (4-8)$$

其中 B 为批大小， α_b 为第 b 个样本的回合级权重。该设计的目的不是改变教师轨迹，而是缓解离线残差误差与闭环任务成功率之间的不完全一致，使纠偏模型更关注成功完成任务的有效行为分布。

本文主线加权配置中，成功回合样本权重设为 2.0，失败回合样本权重设为 0.5；未加权基线则将所有样本权重设为 1.0。该加权只作用于损失层面，不改变各任务集的采样比例，也不对教师轨迹本身做重标定。

4.5 两阶段锚点残差去噪

单锚点方案只执行一次 Action Expert，然后由残差纠偏模型覆盖较长的轨迹跨度。它的优势是调用次数最少，延迟收益最高；局限是当任务对轨迹精度敏感时，单个锚点速度需要外推的区间过长，累计偏移更难由轻量模型完全补偿。两阶段锚点残差去噪针对这一问题引入中间再锚定：先用第一个锚点速度和第一阶段残差纠偏得到中间状态附近的估计，再在中间步重新执行 Action Expert，获得新的锚点速度，最后由第二个残差纠偏模型完成后半段轨迹。

以 10 步去噪为例，两阶段推理可以概括为

$$x_0 \xrightarrow[v_0]{\Delta_\phi^{(1)}} \hat{x}_5, \quad \hat{x}_5 \xrightarrow[v_5]{\Delta_\phi^{(2)}} \hat{x}_{10}, \quad (4-9)$$

其中每一段都遵循式 4-6 的“锚点速度线性外推 + 残差纠偏”形式， $\Delta_\phi^{(1)}$ 与 $\Delta_\phi^{(2)}$ 的上标表示应用于不同区间，而不是两套互不相关的参数。本文当前主线采用共享权重

的两阶段纠偏模型：训练时将 $x_0 \rightarrow x_5$ 与 $x_5 \rightarrow x_{10}$ 两类区间样本混合到同一个数据集中，推理时同一个模型权重可分别作为第一阶段和第二阶段使用，并由起始状态、锚点速度、时间位置和区间配置决定当前纠偏跨度。这样可以用单个模型适配多个阶段，避免为每个阶段单独维护一套纠偏参数。当前主要使用中间步为 5 的配置，用 2 次 Action Expert 调用和 2 次残差纠偏替代原始 10 次 Action Expert 调用。

两阶段方案提供了可配置的去噪激进程度。单锚点策略适合对少步去噪不敏感、或更看重极低延迟的场景；双锚点策略通过增加一次 Action Expert 调用换取更短的纠偏跨度，在复杂任务上更稳健。已有闭环评测显示，双锚点配置在 `libero_goal`、`libero_object` 和 `libero_spatial` 等微调任务上整体已经接近或略高于原始全量去噪模型，在 `libero_10` 困难任务上也与原始模型处于同一量级。因此，继续增加到三阶段或四阶段虽然可以进一步缩短单段纠偏跨度，但每增加一个阶段都会额外引入一次 Action Expert 调用和一次纠偏前向，端到端加速收益会明显下降；在当前任务效果已经基本持平的情况下，本文将两阶段作为质量与延迟之间的主线折中。本文当前实现的是可配置策略，而不是运行时自适应调度器；模型不会在每个评测回合中自动判断任务难度并切换锚点数量。利用 VLM 语义信息动态选择去噪激进程度属于后续工作方向。

4.6 本章小结

本章围绕 VLM + Action Expert 连续动作生成模型提出了锚点残差去噪方法。首先，推理时间拆分表明，Action Expert 虽然小于 VLM 主干，但多步循环使其成为重要时延来源。其次，直接减少去噪步数会在部分 LIBERO 任务上带来明显成功率风险，说明简单截断不足以支撑稳定部署。进一步的速度场分析表明，相邻速度方向高度一致，但累计偏移随步数增长，因此锚点速度可以作为轨迹方向先验，而最终输出仍需要残差纠偏。基于这些观察，本文设计了包含状态分支、上下文压缩分支和残差输出分支的轻量纠偏模型，并通过单锚点与双锚点配置形成不同激进程度的推理路径。下一章将从任务成功率、消融实验和多平台延迟三个角度评估该方法的实际效果。

第五章 实验评测

5.1 实验设置

本章从任务成功率、消融实验、跨模型泛化、跨随机种子稳定性和跨平台推理延迟五个角度评估本文方法。成功率评测使用 LIBERO 基准，包括 `libero_goal`、`libero_spatial`、`libero_object` 和 `libero_10` 四个任务集。除特别说明外，每个任务评测 10 个回合，指标为任务成功率；`libero_10` 的部分 $Pi0.5$ 结果使用 30 个回合/任务，以降低复杂任务上的随机波动。SmolVLA 是本文方法验证的主线模型。

方法对比包含四类配置。第一类是原始 10 步全量去噪，即常规 Action Expert 去噪推理；第二类是 stepN 直接减步配置，即将 Action Expert 调用次数调整为 N 后直接输出，用于步数消融；第三类是单锚点残差纠偏，即执行一次锚点去噪并用残差纠偏模型直接估计最终状态；第四类是双锚点残差纠偏，即将完整去噪轨迹拆为两个纠偏阶段，各阶段共享同一类残差纠偏模型结构，并通过配置控制作用区间。SmolVLA 的 4 个任务集成功率评测还区分未加权双锚点纠偏和加权双锚点纠偏两个训练设置，其中加权纠偏根据回合是否成功在损失层面对样本加权，不改变任务集采样分布。

训练成本方面，本文的加速方法只训练轻量残差纠偏模型，而不重新微调 VLA 主模型或完整蒸馏 Action Expert。以 SmolVLA 的最终加权双锚点配置为例，纠偏模型参数量约为 9.2M，训练使用约 4.2×10^4 条离线教师轨迹采样记录，并在单张 NVIDIA H20 GPU 上约 20 分钟完成。该开销相对完整 VLA 微调较低，因此更适合针对具体任务快速构建低延迟推理配置。

速度测试平台包括 Ascend 310P、AGX Orin 和 AGX Thor。为了方便 LIBERO 成功率测试，本文在校验服务器与端侧运行结果一致的基础上，将任务成功率评测放在 GPU 参考平台上完成。跨平台方法延迟评测均采用 PyTorch 动态图口径，其中 Ascend 310P 通过 `torch_npu` 后端执行；SmolVLA 跨平台单锚点和双锚点结果使用同一 LIBERO 微调模型和相同方法配置。第三章静态图适配实验采用不同微调权重版本，其延迟结果仅用于平台适配收益分析，不纳入本章跨平台动态图加速范围。

5.2 直接减步数实验

直接减步是最直接的推理加速方式。它减少 Action Expert 的调用次数，因此可以直接降低去噪循环开销；但如果模型的少步轨迹偏离全量去噪轨迹，闭环任务成功率会下降。第四章表 4-2 给出 SmolVLA 在四个 LIBERO 任务集代表任务上的 stepN 消融结果，并补充 AGX Thor 上对应步数的动作块延迟。该表使用代表任务均值，不作为完整 4 个任务集主结论。

图 5-1 将表 4-2 的平均成功率和 AGX Thor 延迟放在同一坐标系中。横轴为动作块延迟，纵轴为代表任务平均成功率，点旁数字表示 stepN 配置中的步数，颜色表示相对于 10 步原始推理的加速比。

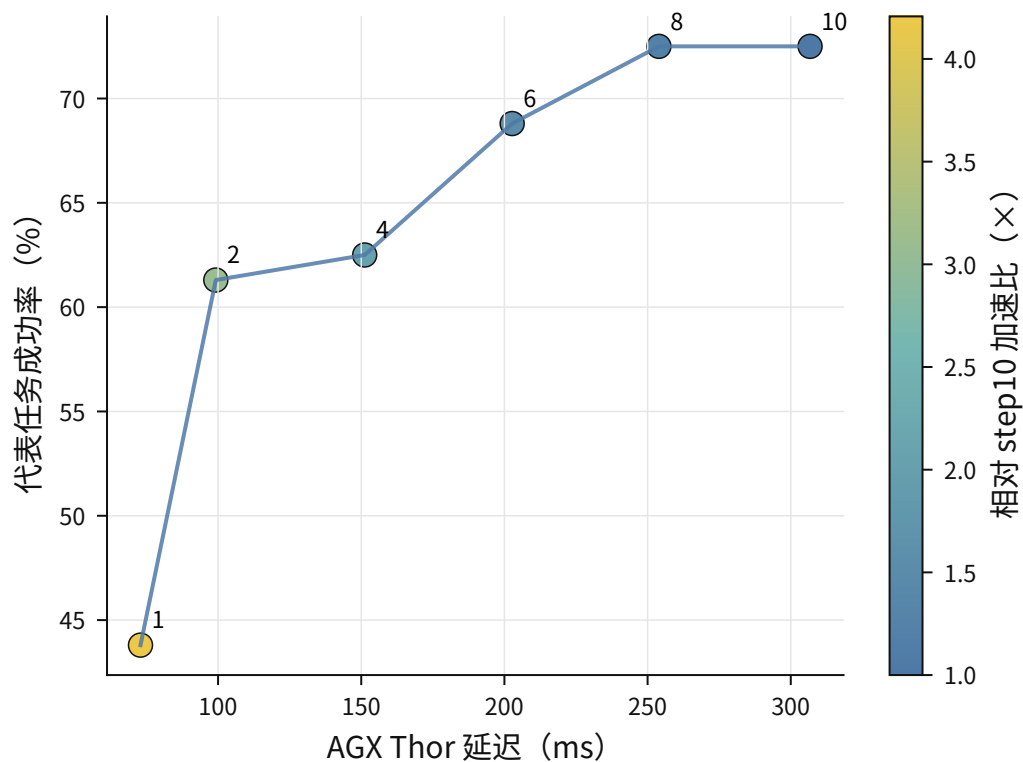


图 5-1 stepN 对推理速度与任务成功率的影响。直接减少 Action Expert 步数虽然能降低时延，但会带来任务质量下降风险。

结果显示，stepN 并不稳定。libero_goal 在 step1 时成功率为 0%，说明一次 Action Expert 调用无法直接替代完整 10 步轨迹；step6 或 step8 虽然可以恢复部分成功率，但端到端速度收益有限。libero_spatial 和 libero_object 对减步更鲁棒，而 libero_10 的结果波动更明显，表明任务难度和长程组合行为会放大少步推

理误差。

5.3 SmolVLA 残差纠偏消融实验

5.3.1 纠偏模型容量

残差纠偏模型需要在少量锚点信息和 VLM 上下文条件下估计全量去噪轨迹的残差。若模型容量过小，纠偏模型难以从视觉语言上下文和动作状态中恢复任务相关误差。表 5-1 比较了两种不同参数规模的单锚点残差纠偏模型在 `libero_goal` 代表任务上的结果。

表 5-1 SmolVLA 残差纠偏模型容量消融

方法	成功率	纠偏模型参数量
原始 10 步	60.0%	—
step1	20.0%	—
单锚点残差纠偏	42.0%	5.3M
单锚点残差纠偏	64.0%	9.2M

约 5.3M 参数的纠偏模型相对 step1 已有一定恢复，但仍低于原始 10 步模型，说明过小的 MLP 或浅层上下文读取结构不足以处理复杂任务中的残差分布。将纠偏模型扩大到约 9.2M 参数后，成功率提高到 64.0%，超过该代表任务上的原始 10 步模型。该结果也是第四章选择较大 SmolVLA 纠偏结构的主要实验依据。

5.3.2 单锚点与双锚点残差纠偏

单锚点方案用一次锚点速度外推到最终状态，延迟最低，但长跨度残差更难拟合。双锚点方案把 10 步轨迹拆为两个较短纠偏区间，需要额外一次 Action Expert 和一次纠偏前向，但降低了每个纠偏区块的跨度。表 5-2 给出 `libero_goal` 全套任务上的对比。

双锚点 ARD 的平均成功率达到 86.0%，比原始 10 步高 5.0 个百分点，也高于单锚点加权方案的 71.0%。其中任务 9 从单锚点的 20% 恢复到 90%，说明长跨度误差在困难任务上会成为主要问题。该结果表明，对于对少步去噪较鲁棒的相对简单任务，单锚点方案可以在保持任务成功率基本稳定的同时获得更高加速收益；而在复杂任务上，单锚点长跨度外推更容易造成成功率下降，需要引入双锚点方案，在保证成

表 5-2 libero_goal 上单锚点与双锚点残差纠偏对比

任务	原始 10 步	step1	单锚点加权	双锚点 ARD	Δ (双锚点 - 原始 10 步)
0	60	0	30	60	0
1	90	100	100	100	+10
2	100	90	90	80	-20
3	30	30	30	50	+20
4	100	60	90	90	-10
5	80	90	80	90	+10
6	80	90	70	100	+20
7	100	100	100	100	0
8	90	100	100	100	+10
9	80	0	20	90	+10
平均	81.0%	66.0%	71.0%	86.0%	+5.0%

功率的同时获得相对保守但更可靠的加速比。

5.3.3 加权训练

加权训练的目标不是改变数据分布，而是在同一批教师轨迹样本上根据回合是否成功重新分配样本损失权重。成功回合被赋予更高权重，失败回合权重较低，使纠偏模型更多拟合已经完成任务的有效行为轨迹。由于加权纠偏与未加权纠偏的差异还受训练随机性和历史模型权重继承关系影响，本文只将其作为经验有效的训练配置，而不把单独的权重设计解释为完全独立的因果结论。

5.4 SmolVLA 4 个任务集成功率评测

表 5-3 汇总了 SmolVLA 在四个 LIBERO 任务集上的成功率结果。与 step1 相比，未加权双锚点纠偏和加权双锚点纠偏均能恢复任务成功率；其中未加权双锚点纠偏的 4 个任务集平均成功率为 74.5%，与原始 10 步的 74.3% 基本持平，加权双锚点纠偏达到 78.0%，相对原始 10 步提升 3.8 个百分点。

图 5-2 对表 5-3 的任务集级成功率进行可视化。step1 在多数任务集上低于原始 10 步，说明直接减步会降低任务质量；未加权双锚点纠偏整体恢复到原始 10 步附近，

表 5-3 SmolVLA 4 个任务集共享纠偏模型成功率汇总

任务集	原始 10 步	step1	未加权双锚点纠偏	加权双锚点纠偏	$\Delta(\text{加权} - \text{原始 10 步})$
libero_spatial	79.0%	77.0%	80.0%	82.0%	+3.0%
libero_goal	80.0%	66.0%	84.0%	88.0%	+8.0%
libero_object	91.0%	68.0%	90.0%	98.0%	+7.0%
libero_10	47.0%	35.0%	44.0%	44.0%	-3.0%
4 个任务集平均	74.3%	61.5%	74.5%	78.0%	+3.8%

加权双锚点纠偏在 libero_goal、libero_object 和 4 个任务集平均值上进一步提升，但在 libero_10 上仍未超过原始 10 步。

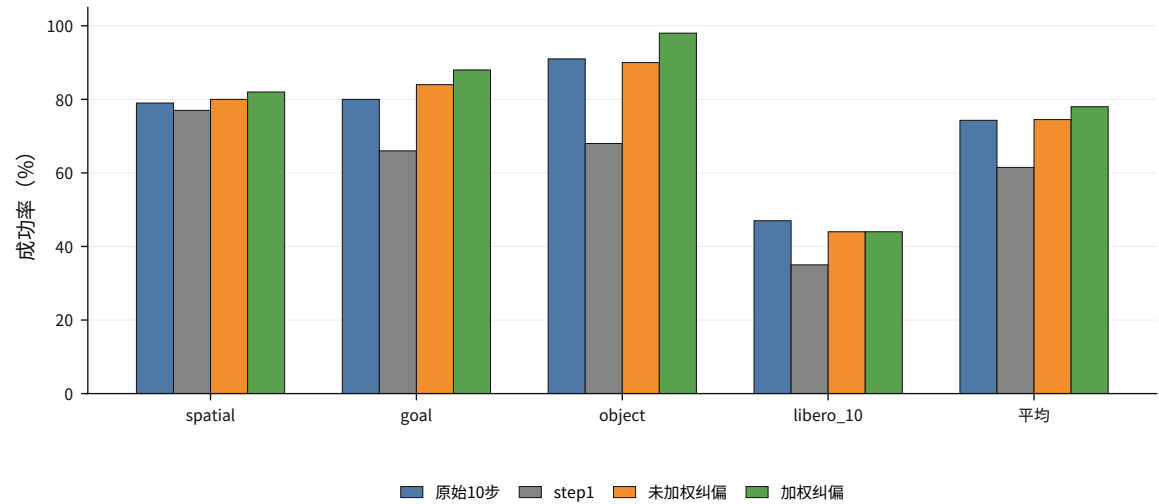


图 5-2 SmolVLA 4 个任务集成功率对比。图中原始 10 步表示全量去噪模型，step1 表示直接单步推理，未加权和加权分别表示未加权双锚点残差纠偏配置，以及根据回合是否成功进行样本加权后的双锚点残差纠偏配置。

这些结果表明，ARD 并不是简单用轻量模型替代 Action Expert，而是在保留原始 Action Expert 锚点能力的基础上补偿少步外推误差。step1 的平均成功率只有 61.5%，说明直接减步会带来任务质量下降；未加权双锚点纠偏恢复到原始 10 步附近，加权双锚点纠偏达到 78.0%，表明残差纠偏可以恢复大部分全量去噪能力，并且加权配置对应更高的平均成功率。

5.5 跨模型验证

为了判断 ARD 是否只对 SmolVLA 有效, 本文补充了 $\pi_{0.5}$ 的 4 个任务集对比结果。表 5-4 显示, $\pi_{0.5}$ 上单锚点残差纠偏的 4 个任务集平均成功率为 89.8%, 相对原始 10 步的 91.1% 下降 1.3 个百分点。

表 5-4 $\pi_{0.5}$ 4 个任务集跨模型补充结果

任务集	原始 10 步	单锚点纠偏	Δ (单锚点 - 原始 10 步)
libero_goal	93.0%	93.0%	0.0%
libero_spatial	96.0%	94.0%	-2.0%
libero_object	91.0%	88.0%	-3.0%
libero_10	84.3%	84.3%	0.0%
4 个任务集平均	91.1%	89.8%	-1.3%

这一结果说明, ARD 并非只对 SmolVLA 生效。SmolVLA 与 $\pi_{0.5}$ 均属于 VLM + Action Expert 连续动作生成结构; 在 SmolVLA 的 LIBERO 4 个任务集评测中, 双锚点残差纠偏可以将平均成功率恢复到原始 10 步模型附近, 加权配置进一步达到 78.0%; 在 $\pi_{0.5}$ 上, 单锚点残差纠偏的 4 个任务集平均成功率为 89.8%, 接近原始 10 步模型的 91.1%。因此, 跨模型结果支持 ARD 对这类 VLA 结构具有可迁移性, 但具体锚点数量和纠偏模型容量仍需结合基座模型、任务难度和延迟预算选择。

5.6 跨随机种子稳定性复验

闭环机器人评测通常存在随机种子、初始物体位置和仿真扰动带来的波动。跨随机种子复验的目的不是强调额外增益, 而是检查双锚点 ARD 是否只适配单一随机种子, 是否会在不同初始状态下出现系统性成功率下降。表 5-5 汇总随机种子 1000、1500 和 3000 三组结果; 为避免与样本加权设置混用, 本节统一采用未加权双锚点 ARD 口径, 只列出该配置成功率以及相对同随机种子原始 10 步模型的变化。

随机种子 1000、1500 和 3000 下, 4 个任务集平均变化分别为 +2.5、+2.0 和 +2.5 个百分点, 未观察到系统性成功率下降。分任务集看, 随机种子 1000 的 libero_10 和随机种子 3000 的 libero_spatial 分别出现 1.0 个百分点的小幅回落, 但其余

表 5-5 SmolVLA 未加权双锚点 ARD 跨随机种子稳定性复验

任务集	随机种子 1000 双锚点	随机种子 1000 变化	随机种子 1500 双锚点	随机种子 1500 变化	随机种子 3000 双锚点	随机种子 3000 变化
libero_goal	86.0%	+5.0%	76.0%	+2.0%	80.0%	+4.0%
libero_spatial	81.0%	+2.0%	81.8%	+2.7%	83.0%	-1.0%
libero_object	95.0%	+4.0%	98.0%	+3.0%	97.0%	+2.0%
libero_10	46.0%	-1.0%	43.8%	+0.4%	52.0%	+5.0%
4 个任务集平均	77.0%	+2.5%	74.9%	+2.0%	78.0%	+2.5%

结果均不低于同随机种子原始 10 步模型。该结果说明，未加权双锚点 ARD 没有只适配单一随机种子；同时，个别任务集的小幅波动仍提示闭环仿真评测需要结合多随机种子结果理解。

5.7 跨平台加速与昇腾验证

5.7.1 Ascend 310P 动态图延迟

表 5-6 给出 Ascend 310P 上 torch_npu 动态图的动作块延迟。该表使用预处理输入的模型核心推理口径，主要用于比较同一模型内原始 10 步、step1 和 ARD 变体的相对变化。由于第三章静态图适配实验与本章方法实验采用不同微调权重版本，二者延迟只在各自实验设置内进行相对比较，不直接合并计算 ARD 加速比。

表 5-6 Ascend 310P torch_npu 动态图动作块延迟

模型	方案	总延迟/ms	相对原始 10 步	VLM 前缀/ms	AE/ms	纠偏/ms
π_0	原始 10 步	467.50	1.00×	296.14	161.62	0.00
π_0	step1	321.49	1.45×	296.69	16.74	0.00
π_0	单锚点残差纠偏	323.66	1.44×	296.71	16.74	1.65
SmolVLA	原始 10 步	377.37	1.00×	121.63	246.39	0.00
SmolVLA	step1	152.94	2.47×	120.56	25.17	0.00
SmolVLA	单锚点残差纠偏	155.20	2.43×	119.74	25.09	1.73
SmolVLA	双锚点 ARD	188.58	2.00×	120.54	50.46	9.62
$\pi_{0.5}$	原始 10 步	606.48	1.00×	365.31	231.86	0.00
$\pi_{0.5}$	step1	397.01	1.53×	365.57	23.31	0.00
$\pi_{0.5}$	单锚点残差纠偏	400.15	1.52×	366.59	21.62	3.38

310P 动态图结果体现出两个特点。第一，SmolVLA 的 Action Expert 是最大可压缩部分，原始 10 步中 AE 为 246.39 ms，双锚点 ARD 后降为 50.46 ms，但 VLM 前缀仍约 120 ms，因此端到端加速被固定前缀成本限制在 2.00× 至 2.43×。第二， π_0 和

$\pi_{0.5}$ 的 VLM 前缀占比更高, step1 或单锚点方案主要压缩 AE, 却无法同步压缩一次性 VLM 成本, 因此同平台端到端加速低于 SmolVLA。若将三种 VLM + Action Expert 模型的 310P 动态图结果共同纳入摘要口径, π_0 单锚点残差纠偏的 $1.44\times$ 构成跨模型延迟收益下限; SmolVLA 的三平台结果则用于说明同一微调模型和相同 ARD 配置在不同端侧硬件上的迁移收益。

5.7.2 端侧平台对比

表 5-7 汇总 SmolVLA 在 Ascend 310P、AGX Orin 和 AGX Thor 上的动作块端到端延迟。三组平台使用同一 SmolVLA LIBERO 微调模型和相同 ARD 配置, 均以各平台 PyTorch 动态图原始 10 步结果为基线计算加速比。不同平台的绝对延迟仍会受到硬件运行时、后端实现和计时脚本影响, 因此该表用于展示方法在不同端侧平台上的迁移收益, 而不是比较硬件峰值性能。

表 5-7 SmolVLA 跨平台动作块延迟对比

平台	原始 10 步/ms	单锚点/ms	单锚点加速	双锚点/ms	双锚点加速
Ascend 310P	377.37	155.20	$2.43\times$	188.58	$2.00\times$
AGX Orin	1399.97	343.91	$4.07\times$	479.13	$2.92\times$
AGX Thor	307.63	74.74	$4.12\times$	106.14	$2.90\times$

图 5-3 对表 5-7 中的加速比进行可视化。单锚点方案在三个平台上均取得更高速度收益, 双锚点方案则以额外一次锚点去噪换取更稳健的轨迹质量; 两种配置在不同平台上的相对趋势保持一致。

结果表明, ARD 的延迟收益在不同端侧平台上均能体现。Orin 和 Thor 的 PyTorch 动态图中, Action Expert 多步循环开销占比较高, 因此单锚点方案可达到约 $4.0\times$, 双锚点方案约 $2.9\times$ 。310P 动态图上, VLM 前缀与运行时固定成本占比更高, 双锚点方案的端到端加速为 $2.00\times$ 。这并不说明 310P 上纠偏无效, 而是反映当 AE 调用次数被压缩后, 固定 VLM 前缀成为新的端到端限制。

5.7.3 昇腾静态图部署链路补充分析

前两张表关注同一动态图库内的延迟变化: 表 5-6 给出跨模型 310P 动态图结果, 表 5-7 给出同一 SmolVLA 微调模型的跨平台结果。为了补充说明 Ascend 310P 上静态图部署链路能够带来的工程收益, 表 5-8 单独给出另一微调权重版本下从 PyTorch

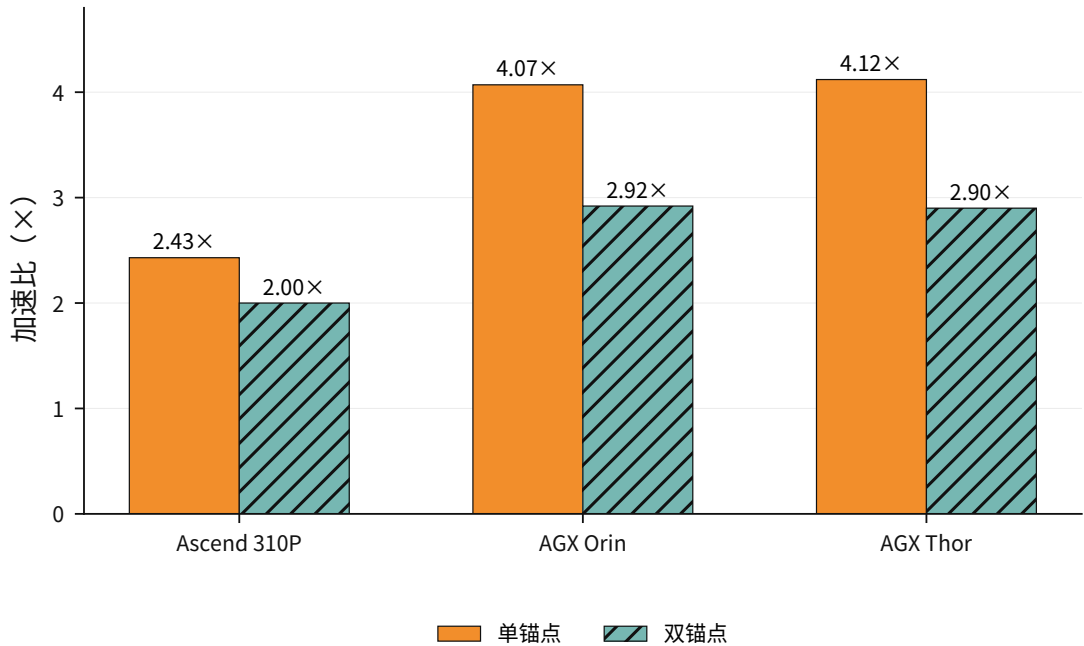


图 5-3 SmolVLA 跨平台 ARD 端到端加速比。图中加速比均以各平台 PyTorch 动态图原始 10 步推理为基线；单锚点和双锚点分别对应表 5-7 中的单锚点残差纠偏与双锚点 ARD 配置。

原始动态图到最终静态图 + ARD 方案的收益拆解。该表不是跨平台方法消融，也不纳入摘要中 1.44x 至 4.12x 的动态图加速范围；它反映的是静态图转换、fp16 与算子兼容性修复、模型拆分、OM 串联运行以及减少 Action Expert 调用次数共同作用后的部署链路收益。

表 5-8 Ascend 310P 静态图部署链路补充收益拆解

模型	最终方案	PyTorch 原始/ms	静态原始/ms	最终延迟/ms	静态图收益	方法收益	总收益
π_0	单锚点残差纠偏	467.50	294.369	192.423	1.59×	1.53×	2.43×
SmolVLA	单锚点残差纠偏	488.34	147.290	80.123	3.32×	1.84×	6.09×
SmolVLA	双锚点 ARD	488.34	147.290	86.137	3.32×	1.71×	5.67×
$\pi_{0.5}$	单锚点残差纠偏	945.00	410.200	240.384	2.30×	1.71×	3.93×

表中收益均以倍数表示。“静态图收益”表示原始动态图到原始静态图链路的优化；“方法收益”表示在静态图链路内部引入轻量纠偏后进一步减少 Action Expert 调用的收益；“总收益”表示最终方案相对该微调权重版本 PyTorch 原始动态图的整体收益。SmolVLA 单锚点和双锚点的最终延迟分别为 80.123 ms 和 86.137 ms，在该静态图部署链路中相对 PyTorch 原始动态图分别达到 6.09x 和 5.67x。 $\pi_{0.5}$ 单锚点为 3.93x。

π_0 行的 PyTorch 原始延迟采用表 5-6 的同口径动态图基线；当前使用单锚点残差纠偏进行延迟核算，且 AE 静态图不直接输出真实隐藏状态，因此该行只作为延迟边界，不作为动作正确性结论。

除延迟外，本文还对 Ascend 310P 静态图链路做了正确性验证。静态图输出与参考 GPU 运行结果在关键输出上保持高一致性，代表性输出余弦相似度高于 99.00%；在 LIBERO 仿真实验中，310P 静态图链路的运行结果与参考平台一致。该结果表明，第三章提出的转换、自动校验和仿真回放流程能够支撑后续加速实验，而不是只得到离线张量基准。

5.8 分析与讨论

综合上述结果，本文方法的有效性主要体现在三点。第一，stepN 直接减步虽然可以降低延迟，但在 SmolVLA 上会带来明显成功率损失，尤其是 libero_goal 和 libero_10 等较困难任务。第二，ARD 利用原始 Action Expert 的锚点速度和 VLM 上下文，只学习少步外推到全量轨迹之间的残差，因此比 step1 更稳定；双锚点方案进一步降低长跨度残差难度，在成功率和延迟之间取得更好的折中。第三，端侧延迟结果表明，Action Expert 多步循环确实是可压缩对象，但压缩后固定 VLM 前缀、运行时调度和数据搬运会成为新的端到端限制。

libero_10 是当前方法的主要边界。该任务集的任务往往包含更长任务链、更复杂物体关系和更强的初始状态敏感性。即使速度场相邻方向较一致，也不代表少步轨迹一定能保持闭环任务成功率，因为接触时机、夹爪状态和中间物体位置可能对局部动作误差高度敏感。因此，libero_10 上未超过原始 10 步模型更合理的解释是任务本身更困难、当前蒸馏方法效果有限；训练覆盖和样本权重只能作为补充因素，不能作为唯一原因。

平台结果也说明，端到端加速不能只看纠偏模型本身的耗时。纠偏模型通常只有数毫秒，但它能否转化为整体加速取决于被替代的 Action Expert 调用成本以及固定 VLM 前缀的占比。在跨平台动态图实验中，Orin 和 Thor 的 Action Expert 循环开销较高，因此同一类方法呈现更大的端到端加速比；在补充的 Ascend 310P 静态图链路中，Action Expert 已经被 OM 和混合精度优化压缩，单锚点和双锚点方案的端到端上限转而受 VLM 部分限制。后续若继续提升端到端速度，需要同时考虑 VLM 前缀压缩、输入预处理和运行时调度优化。

5.9 本章小结

本章通过 LIBERO 成功率、消融实验、跨模型验证、跨随机种子复验和多平台延迟评测评估了本文方法。实验表明, SmolVLA 的 step1 会将 4 个任务集平均成功率降至 61.5%, 而共享残差纠偏模型可恢复至 74.5%, 加权配置达到 78.0%, 高于原始 10 步的 74.3%。跨随机种子复验显示, 随机种子 1000、1500 和 3000 的未加权双锚点结果均未出现系统性成功率下降, 说明该方法没有只适配单一随机种子。延迟方面, 在 Ascend 310P 动态图口径下, π_0 单锚点残差纠偏为 1.44 $\times$, SmolVLA 双锚点/单锚点 ARD 为 2.00 $\times$ /2.43 $\times$, $\pi_{0.5}$ 单锚点残差纠偏为 1.52 $\times$; 基于同一 SmolVLA LIBERO 微调模型的跨平台实验进一步显示, 单锚点配置在 Ascend 310P、AGX Orin 和 AGX Thor 上达到 2.43 $\times$ 至 4.12 $\times$ 端到端加速, 双锚点配置达到 2.00 $\times$ 至 2.92 $\times$ 端到端加速。Ascend 310P 静态图部署链路则作为补充分析说明最终工程方案的系统收益。综合来看, 锚点残差去噪可以在保持任务成功率的同时降低 VLA 连续动作模型的推理开销。

第六章 总结与展望

6.1 主要工作总结

本文围绕视觉-语言-动作 (Vision-Language-Action, VLA) 模型的昇腾算力适配与优化展开研究, 目标是在国产端侧平台上验证主流具身智能模型的部署可行性, 并进一步降低连续动作生成模型的推理时延。全文工作可以概括为以下四个方面^①。

第一, 本文完成了面向 Ascend 310P 的 VLA 与具身策略模型迁移流程。针对 PyTorch 动态图、ONNX 导出、ATC 编译、OM 静态图推理、混合精度配置和数值正确性验证等环节, 本文整理了可复用的适配路径, 并结合多个模型的实际调试过程分析了算子兼容、动态张量形状、精度漂移和推理接口统一等问题。该部分工作不仅关注单个模型能否运行, 也强调将适配过程整理为后续模型迁移可以复用的工程方法。

第二, 本文围绕静态图部署和混合精度优化评估了昇腾平台上的推理收益。实验覆盖 ACT、Diffusion Policy、SmolVLA、OpenVLA 等不同结构的六组模型, 比较了原始动态图路径与 Ascend 310P 静态图路径的延迟变化, 并通过成功率一致性与关键输出余弦相似度验证迁移后模型行为的正确性。该部分结果表明, VLA 及相关具身策略模型在 Ascend 310P 上具备形成稳定部署链路的基础。

第三, 本文针对 VLM + Action Expert 结构中的多步去噪瓶颈提出锚点残差去噪 (Anchor-Residual Denoising, ARD) 方法。通过对 SmolVLA 及其他 VLM + Action Expert 模型的分项延迟分析, 本文发现 Action Expert 多步循环是连续动作生成模型中的主要可优化部分; 直接减少去噪步数虽然能够降低延迟, 但会带来成功率下降。为此, 本文利用少量锚点去噪步骤保留原始 Action Expert 的轨迹方向先验, 并引入轻量残差纠偏模型补偿少步外推与全量去噪轨迹之间的偏差。纠偏模型进一步利用 VLM 上下文, 并根据回合是否成功进行样本加权, 使少步推理在保持速度收益的同时尽量恢复任务质量。

第四, 本文从 LIBERO 成功率和多平台延迟两个角度验证了方法效果。成功率方面, 本文以 SmolVLA 为主线, 在 LIBERO 4 个任务集、跨随机种子复验和跨模型实验中评估 stepN 直接减步、单锚点残差纠偏和双锚点残差纠偏等配置。延迟方面, 本文在 Ascend 310P、AGX Orin 和 AGX Thor 上基于同一 SmolVLA LIBERO 微调模型

^① 本章部分总结性语句曾使用 DeepSeek V4 辅助进行语言凝练, 修改后的技术表述、实验数据和结论均由作者人工核验。

比较原始全量去噪与少步纠偏方案的动态图动作块推理耗时，并在 Ascend 310P 上进一步补充静态图部署链路收益分析。

6.2 主要结论

本文的实验结果支持以下结论。

首先，主流 VLA 与具身策略模型可以在 Ascend 310P 上形成可验证的部署路径。在六组模型适配案例中，静态图优化后的平均推理延迟由 1049.80 ms 降至 360.15 ms，整体加速比约为 2.91 $\times$ 。同时，在已完成任务级验证的代表性任务上，Ascend 310P 与 GPU 运行结果保持一致，关键输出的余弦相似度高于 99.00%。这些结果表明，静态图转换、混合精度优化和针对性算子修复能够在多数模型上带来实际延迟收益，并且不会破坏本文评测口径下的任务行为正确性。

其次，VLM + Action Expert 模型的推理加速不能简单等同于减少去噪步数。以 SmolVLA 为例，Action Expert 在 Ascend 310P、AGX Orin 和 AGX Thor 上分别占动作块延迟的 65.3%、74.2% 和 85.1%，说明多步去噪循环确实是主要优化对象；但直接将 SmolVLA 减少到单步去噪时，4 个任务集平均成功率仅为 61.5%，低于原始全量去噪模型的 74.3%。因此，少步推理必须同时处理速度收益和轨迹质量恢复两个问题。

再次，锚点残差去噪能够在减少 Action Expert 调用次数的同时保持较高任务成功率。在 SmolVLA 的 4 个任务集平均口径下，原始全量去噪模型、未加权双锚点纠偏方案和加权双锚点纠偏方案的平均成功率分别为 74.3%、74.5% 和 78.0%；跨随机种子复验中，随机种子 1000、1500 和 3000 的未加权双锚点结果均未出现系统性成功率下降，4 个任务集平均结果分别高于同随机种子原始 10 步模型 2.5、2.0 和 2.5 个百分点。该结果支持利用原始 Action Expert 的锚点速度场并学习残差补偿，而不是单纯直接减步，来实现连续动作 VLA 的低延迟推理。

最后，本文方法在端侧硬件上呈现一致的延迟收益，但端到端加速比受固定 VLM 前缀开销影响。动态图延迟评测显示，在 Ascend 310P 跨模型实验中，单锚点或双锚点 ARD 配置获得 1.44 $\times$ 至 2.43 $\times$ 的端到端加速；在同一 SmolVLA LIBERO 微调模型和相同方法配置下，Ascend 310P、AGX Orin 和 AGX Thor 上的跨平台加速范围进一步达到 2.00 $\times$ 至 4.12 $\times$ 。综合跨模型与跨平台动态图实验，本文方法的端到端加速范围为 1.44 $\times$ 至 4.12 $\times$ 。补充的 Ascend 310P 静态图部署链路结果表明，静态图转换与 ARD 可以进一步形成系统级工程收益，但该部分采用不同微调权重版本，不纳入上述动态图加速范围。总体上，ARD 主要压缩可重复执行的 Action Expert 部分，而端

到端系统仍需要继续优化 VLM 前缀、预处理和运行时调度等固定开销。

6.3 局限性

本文仍存在以下局限。

第一，任务评测范围仍以 LIBERO 仿真基准为主，尚未完成真实机器人闭环验证。LIBERO 能够提供标准化的成功率比较和较高复现实验效率，但仿真任务不能完全覆盖真实机械臂中的感知噪声、执行误差、时延抖动和物理接触不确定性。因此，本文关于任务成功率的结论应理解为仿真闭环评测结果，而不是对真实机器人部署效果的直接保证。

第二，当前蒸馏方法在更困难任务组合上仍有不足。特别是在 `libero_10` 上，加权残差纠偏方案未能超过原始全量去噪模型，说明该任务集中更长程的任务结构、多物体交互和更复杂的状态转移会放大少步外推误差。本文通过回合成功标记加权缓解这一问题，但当前蒸馏目标仍主要拟合全量去噪轨迹的动作输出，对失败原因、接触阶段和长程子目标的建模还不充分。

第三，模型覆盖和工程链路仍有边界。本文以 SmolVLA 作为方法验证主线，其他连续动作 VLA 模型主要用于结构迁移、分项延迟和跨模型现象分析。另一方面，Ascend 310P 静态图部署仍依赖具体模型结构、输入张量形状和 CANN 工具链状态，部分算子兼容和精度配置仍需要人工分析。当前 ARD 也采用固定的单锚点或双锚点配置，尚未根据任务难度和运行时状态自适应选择去噪预算。

6.4 未来工作

后续工作可以从三个方向继续推进。

第一，进一步扩展评测基准和真实系统验证。本文当前的成功率评测主要集中在 LIBERO 系列标准任务上，后续可以扩展到 LIBERO-PRO 等更强调扰动、泛化和长程组合能力的仿真基准^[31]，并补充真实机械臂闭环实验。更完整的评测应同时覆盖不同随机种子、不同任务难度、感知扰动和真实执行时延，从而判断少步纠偏方法在开放环境中的稳定性。

第二，发展运行时自适应的去噪预算调度。当前 ARD 使用预设的单锚点或双锚点配置，尚未根据任务状态动态调整推理预算。未来可以利用 VLM 语义表示、动作不确定性、历史执行反馈和硬件实时负载估计任务难度，使简单任务采用更激进的

去噪压缩，而困难任务保留更多锚点或回退到全量去噪。同时，可以继续改进训练目标，将接触关键帧、失败样本、动作关键维度和长程子目标显式纳入损失设计，以提升 libero_10 这类困难任务上的蒸馏效果。

第三，将适配与加速方法扩展到更多模型和平台。本文验证的模型主要覆盖约 0.6B 到 3B+ 规模的 VLM + Action Expert 连续动作模型，未来可以进一步考察 GR00T N1 这类 VLM + diffusion transformer 双系统模型^[32]，以及 DreamZero 14B 等更大规模的机器人基础模型^[33]。同时，昇腾适配流程也可以继续沉淀为更自动化的算子诊断、精度回归和端到端延迟分析工具，使国产端侧算力上的 VLA 部署从个案迁移走向更稳定的工程流程。

6.5 本章小结

本章总结了本文围绕 Ascend 310P 平台适配、锚点残差去噪方法设计、LIBERO 成功率验证和多平台延迟评测开展的工作，归纳了主要实验结论，并说明了当前方法在任务覆盖、困难任务泛化、模型范围和工程自动化方面的不足。总体而言，本文验证了 VLA 模型在国产端侧算力上的部署可行性，也表明面向 Action Expert 多步去噪循环的少步纠偏是降低连续动作 VLA 推理时延的一条可行路径。

参 考 文 献

- [1] 白辰甲, 许华哲, 李学龙. 大模型驱动的具身智能: 发展与挑战[J]. 中国科学: 信息科学, 2024, 54: 2035-2082. DOI: 10.1360/SSI-2024-0076.
- [2] 王文晟, 谭宁, 黄凯, 等. 基于大模型的具身智能系统综述[J]. 自动化学报, 2025, 51(1): 1-19. DOI: 10.16383/j.aas.c240542.
- [3] BROHAN A, BROWN N, CARBAJAL J, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[J/OL]. ArXiv preprint arXiv:2307.15818, 2023. <https://arxiv.org/abs/2307.15818>.
- [4] KIM M J, PERTSCH K, KARAMCHETI S, et al. OpenVLA: An Open-Source Vision-Language-Action Model[J/OL]. ArXiv preprint arXiv:2406.09246, 2024. <https://arxiv.org/abs/2406.09246>.
- [5] BLACK K, BROWN N, DRIESS D, et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control[J/OL]. ArXiv preprint arXiv:2410.24164, 2024. <https://arxiv.org/abs/2410.24164>.
- [6] Physical Intelligence, BLACK K, BROWN N, et al. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization[J/OL]. ArXiv preprint arXiv:2504.16054, 2025. <https://arxiv.org/abs/2504.16054>.
- [7] SHUKOR M, AUBAKIROVA D, CAPUANO F, et al. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics[J/OL]. ArXiv preprint arXiv:2506.01844, 2025. <https://arxiv.org/abs/2506.01844>.
- [8] 兰沅卜, 赵文博, 朱凯, 等. 基于具身智能的移动操作机器人系统发展研究[J]. 中国工程科学, 2024, 26(1): 139-148. DOI: 10.15302/J-SSCAE-2024.01.010.
- [9] 章晋睿, 龙婷婷, 张德宇, 等. 端智能推理加速技术综述[J]. 电子学报, 2025, 53(4): 1063-1102. DOI: 10.12263/DZXB.20240691.
- [10] ZHOU K, CHEN Q, PENG D, et al. Characterizing Vision-Language-Action Models across XPU: Constraints and Acceleration for On-Robot Deployment[C]//International Conference on Machine Learning. 2026.
- [11] CHI C, FENG S, DU Y, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion[C/OL]//Proceedings of Robotics: Science and Systems. 2023. <https://arxiv.org/abs/2303.04137>.
- [12] LIPMAN Y, CHEN R T Q, BEN-HAMU H, et al. Flow Matching for Generative Modeling[C/OL]//International Conference on Learning Representations. 2023. <https://arxiv.org/abs/2210.02747>.
- [13] SALIMANS T, HO J. Progressive Distillation for Fast Sampling of Diffusion Models[C/OL]//International Conference on Learning Representations. 2022. <https://arxiv.org/abs/2202.00512>.
- [14] LEVIATHAN Y, KALMAN M, MATIAS Y. Fast Inference from Transformers via Speculative Decoding[C/OL]//Proceedings of the 40th International Conference on Machine Learning. 2023. <https://proceedings.mlr.press/v202/leviathan23a.html>.
- [15] LI Y, WEI F, ZHANG C, et al. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test[C/OL]//Advances in Neural Information Processing Systems. 2025. <https://openreview.net/forum?id=4exx1hUffq>.
- [16] ZHOU K, YAN X, WEI X, et al. Multimodal Generation Made Efficient with Approximate Speculative Diffusion and Speculative Verification[C/OL]//Advances in Neural Information Processing Systems. 2025. <https://neurips.cc/virtual/2025/poster/118825>.

- [17] Huawei. ATC Tool: Converting an ONNX Model to an OM Model[EB/OL]. 2026 [2026-04-26]. https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/850alpha002/graph/graphdevg/atlasag_25_0080.html.
- [18] Huawei Technologies Co., Ltd. PyTorch Adapter for Ascend NPU[EB/OL]. 2026 [2026-04-26]. <https://gitee.com/ascend/pytorch>.
- [19] ZHAO T Z, KUMAR V, LEVINE S, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware[C/OL]// Proceedings of Robotics: Science and Systems. 2023. <https://arxiv.org/abs/2304.13705>.
- [20] KIM M J, FINN C, LIANG P. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success[J/OL]. ArXiv preprint arXiv:2502.19645, 2025. <https://arxiv.org/abs/2502.19645>.
- [21] LIU S, WU L, LI B, et al. RDT-1B: A Diffusion Foundation Model for Bimanual Manipulation[J/OL]. ArXiv preprint arXiv:2410.07864, 2024. <https://arxiv.org/abs/2410.07864>.
- [22] LIU X, GONG C, LIU Q. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow[C/OL]// International Conference on Learning Representations. 2023. <https://arxiv.org/abs/2209.03003>.
- [23] SONG Y, DHARIWAL P, CHEN M, et al. Consistency Models[C/OL]// Proceedings of the 40th International Conference on Machine Learning. 2023: 32211-32252. <https://proceedings.mlr.press/v202/song23a.html>.
- [24] MIAO X, OLIARO G, ZHANG Z, et al. SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification[C/OL]// Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2024. <https://arxiv.org/abs/2305.09781>.
- [25] CAI T, LI Y, GENG Z, et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads[C/OL]// Proceedings of the 41st International Conference on Machine Learning. 2024: 5209-5235. <https://proceedings.mlr.press/v235/cai24b.html>.
- [26] LI Y, WEI F, ZHANG C, et al. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty[C/OL]// Proceedings of the 41st International Conference on Machine Learning. 2024. <https://proceedings.mlr.press/v235/li24bt.html>.
- [27] LI Y, WEI F, ZHANG C, et al. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees[J/OL]. ArXiv preprint arXiv:2406.16858, 2024. <https://arxiv.org/abs/2406.16858>.
- [28] JI K, MENG Y, CUI H, et al. Block-wise Adaptive Caching for Accelerating Diffusion Policy[C/OL]// International Conference on Learning Representations. 2026. <https://openreview.net/forum?id=c6ZWfQLOWD>.
- [29] LI Y, FENG J, MENG Y, et al. TS-DP: Reinforcement Speculative Decoding For Temporal Adaptive Diffusion Policy Acceleration[J/OL]. ArXiv preprint arXiv:2512.15773, 2025. <https://arxiv.org/abs/2512.15773>.
- [30] LIU B, ZHU Y, GAO C, et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning[C/OL]// Advances in Neural Information Processing Systems. 2023. <https://arxiv.org/abs/2306.03310>.
- [31] ZHOU X, XU Y, TIE G, et al. LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization[J/OL]. ArXiv preprint arXiv:2510.03827, 2025. <https://arxiv.org/abs/2510.03827>.
- [32] NVIDIA, BJORCK J, CASTANEDA F, et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots[J/OL]. ArXiv preprint arXiv:2503.14734, 2025. <https://arxiv.org/abs/2503.14734>.

- [33] YE S, GE Y, ZHENG K, et al. World Action Models are Zero-shot Policies[J/OL]. ArXiv preprint arXiv:2602.15922, 2026. <https://arxiv.org/abs/2602.15922>.

符号与缩写（附录 1）

1.1 主要符号与缩写

表 1-1 主要符号与缩写说明

符号或缩写	含义
VLA	Vision-Language-Action Model, 视觉-语言-动作模型
VLM	Vision-Language Model, 视觉语言模型
AE	Action Expert, 动作专家模块
ARD	Anchor-Residual Denoising, 锚点残差去噪
OM	昇腾 CANN 工具链生成的离线模型格式
ATC	Ascend Tensor Compiler, 昇腾模型转换工具
x_i	第 i 个去噪步的内部动作状态
v_i	Action Expert 在第 i 个去噪步预测的速度场
c	VLM 前缀产生的视觉语言条件上下文
Δ_ϕ	轻量残差纠偏模型
stepN	直接执行 N 次 Action Expert 去噪后输出的消融配置

AI 工具使用说明（附录 2）

毕业设计（论文）标题：VLA 模型在昇腾算力上的适配与优化
学生姓名：陈启炜 学号：522031910299
指导教师：古金宇

一、本文使用的 AI 工具清单

序号	AI 工具名称	版本/ 型号	毕业设计（论文） 中应用部分	人为判断过程说明
1	DeepSeek	R1	A2	通过提示词要求给出具体文献来源，对每一条文献的真实性和准确性进行人工校验。
2	DeepSeek	R1	E1	通过提示词要求模型将参考文件按照规定格式整理总结，人工校验参考文献的来源、真实性和相关性。
3	DeepSeek	V4	D3	通过提示词要求对部分句子表达进行凝练总结，使语句表达更为通顺流畅，对修改后句子进行人工校验。

二、使用说明

毕业设计（论文）中应用部分对照表如下：

- A.** 研究准备阶段：A1. 选题与方向设定；A2. 文献检索与筛选。
- B.** 研究设计阶段：B1. 研究思路设计；B2. 方法论构建；B3. 研究工具开发。
- C.** 研究实施阶段：C1. 实验/数据分析；C2. 代码编写与调试；C3. 数学建模与计算。
- D.** 论文写作阶段：D1. 大纲与逻辑优化；D2. 论文正文写作；D3. 语言与语法润色。
- E.** 其他辅助环节：E1. 参考文献与格式；E2. 图表/多媒体生成；E3. 学术反馈与模拟答辩。

学术诚信声明

本人承诺：论文核心观点、实验数据及结论均独立完成，AI 工具仅用于辅助性工作，已明确说明所有生成内容并在文中对应位置进行脚注标注。

承诺人：
日期：2026年5月25日

攻读学位期间学术论文和科研成果目录

- [1] Kaijun Zhou, Qiwei Chen, Da Peng, Zhiyang Li, Xijun Li, Jinyu Gu. Characterizing Vision-Language-Action Models across XPUs: Constraints and Acceleration for On-Robot Deployment. International Conference on Machine Learning, 2026. (第二作者, 见参考文献^[10])

致 谢

行笔于此，已是丙午年初夏，本科时光渐至终章。四年间，遇良师益友，见 AI 时代巨变，褪去几分青涩，收获些许沉稳。回首来路，心中满溢感激。

椿萱并茂，寸草春晖。感谢父母一如既往的陪伴与支持，你们始终如一的支持与鼓励，是我前行最深的底气与动力。

幸遇明师，传道解惑。感谢导师古金宇副教授的言传身教与耐心指引，也感谢实验室诸位老师的授业解惑，令我于学业之中渐有所悟。

同门共读，疑义相析。感谢周楷竣、彭达学长的倾力指导与真诚交流，也谢谢实验室各位同窗的陪伴与帮助，让这条探索之路不再孤单。

寒窗四载，挚友在侧。感谢那些一起做项目、一起熬夜、一起分享焦虑与喜悦的同学们。因为有你们，这段时光才如此充盈而难忘。

行文将尽，而前路方长。谢谢所有曾予我帮助与善意的人，恕不能一一尽数，但每一份我都记在心里。愿我们此去，各有山海，各自灿烂。

ADAPTATION AND OPTIMIZATION OF VISION-LANGUAGE-ACTION MODELS ON ASCEND COMPUTING PLATFORMS

Vision, language, and action (VLA) models integrate visual perception, language understanding, and robot action generation into a unified policy framework. With the development of RT-2, OpenVLA, π_0 , $\pi_{0.5}$, SmolVLA, and related models, robot policies are gradually moving from task specific controllers to general models that understand open language instructions and generate continuous actions. However, deployment is still difficult. Most implementations depend on CUDA and PyTorch eager execution, so migration to Ascend edge hardware requires operator repair, interface alignment, dynamic graph and static graph optimization, mixed precision configuration, and task level validation. In addition, many continuous action VLA models use a Vision Language Model (VLM) together with an Action Expert (AE). The VLM constructs visual and language conditions, while the AE repeatedly performs denoising or velocity field prediction to produce an action chunk, creating a latency bottleneck in closed loop control.

This thesis studies the adaptation and optimization of VLA models on Ascend computing platforms. It has two connected goals: verifying that representative VLA and embodied policy models can run correctly on Ascend 310P, and reducing the latency of VLM + Action Expert models without simply discarding denoising steps. For platform adaptation, this thesis builds a workflow covering PyTorch dynamic graph inference, ONNX export, ATC compilation, OM static graph inference, operator repair, mixed precision optimization, and end to end correctness validation. Across six model adaptation cases, static graph optimization reduces the average inference latency from 1049.80 ms to 360.15 ms. On representative LIBERO, ALOHA, and PushT tasks, Ascend 310P matches the NVIDIA GPU reference in task success rate, and the cosine similarity of output action vectors is above 99.00%.

The inference acceleration part focuses on the repeated AE denoising loop. A straightforward method is stepN, which directly reduces the number of AE calls. This does reduce computation, but robot control is sensitive to small action errors and direct truncation changes the trajectory integration process. Experiments show that aggressive step reduction may reduce

success rate on LIBERO tasks. Therefore, this thesis proposes Anchor Residual Denoising (ARD), which rewrites the original multi step denoising process as a small number of original AE anchor calls plus lightweight residual correction. The key point is not that the small correction model is stronger than the original AE, nor that it replaces the whole denoising model from scratch. ARD reuses the trajectory direction predicted by the original AE at a few anchor states, and trains a small model to correct the accumulated deviation between low step extrapolation and the full teacher denoising trajectory. In this formulation, the original AE still provides the main velocity field and action prior, while the residual model only learns a local correction problem.

More specifically, consider an original denoising process with $N = 10$ AE calls. Starting from an action state x_0 , the AE predicts a velocity v_i at each step according to the current state, time step, and VLM condition context, and the action state is then updated. If only the first velocity v_0 is used for linear extrapolation, computation is reduced but the changes of the velocity field along the full trajectory are ignored. ARD keeps this extrapolation as a strong prior and learns the missing residual. For a correction block starting from x_s and covering K denoising steps, the method calls the original AE once to obtain the anchor velocity v_s , extrapolates from x_s and v_s , and uses the corresponding full trajectory state x_{s+K}^T as the teacher. The residual target is the difference between the teacher state and the extrapolated state. A lightweight model Δ_ϕ receives the anchor state, anchor velocity, time position, and compressed VLM context, and predicts this residual. At inference time, the predicted residual is added back to the extrapolated state to obtain the corrected action state.

ARD supports different anchor configurations. In the single anchor configuration, the model performs one original AE call at x_0 , extrapolates toward the final denoising state, and applies one residual correction forward pass. This gives the lowest latency because 10 AE calls are replaced by one AE call and one small correction call, but the residual model must cover a long interval. In the two-anchor configuration used as the main quality latency tradeoff, the interval is split into two shorter correction blocks. The first block calls the original AE at x_0 , extrapolates to the neighborhood of step 5, and applies residual correction to obtain $\hat{x}_5$. The second block calls the original AE again at $\hat{x}_5$, obtains a new anchor velocity, and applies another extrapolation and residual correction to obtain $\hat{x}_{10}$. Thus, the original 10 AE calls are replaced by two original AE calls and two lightweight correction forwards. This

keeps more of the original denoising trajectory structure than direct step reduction while still removing most repeated AE calls.

The design is supported by velocity field analysis. Adjacent velocity directions are highly consistent, so an anchor velocity can provide the main trajectory direction, but long range extrapolation still accumulates deviation. This explains why ARD restricts the learning target to the residual space. The correction model is not asked to understand the entire visual language instruction or imitate all 10 AE calls. It only compensates for the difference between an anchor based approximation and the teacher trajectory, which reduces the training burden and makes a lightweight model sufficient for practical acceleration.

The residual model still needs task semantics because the correct correction depends on the image, language instruction, robot state, and task phase. Directly feeding the full VLM prefix KV cache into a small model would create large memory and computation overhead, so this thesis designs a compressed context branch. For SmolVLA, the VLM prefix KV cache is projected and mixed across layers to obtain a smaller set of context tokens. Learned queries then read these tokens through cross attention and produce a compact visual language condition vector. In parallel, a state branch encodes the anchor state x_s , anchor velocity v_s , and time position. The context and state representations are fused by an MLP, and the model outputs a normalized residual with the same shape as the action chunk. The main SmolVLA correction model has about 9.237M parameters, which is lightweight compared with the fine tuned SmolVLA checkpoint used in this work, whose measured scale is about 0.605B, including about 507M VLM parameters and about 97M AE parameters.

ARD is trained from full denoising teacher trajectories. For each sample, the system records denoising states, velocities, time steps, VLM prefix KV cache, prefix mask, task metadata, and rollout result. The main SmolVLA training set covers LIBERO 4-suite tasks 0–3, with 2×10 trajectory samples per task and a 3:1 train validation split. For each correction block, the training pipeline constructs the anchor state, anchor velocity, teacher state, and residual target. The base loss is residual mean squared error after normalization. Because offline residual error is not perfectly aligned with closed loop success rate, this thesis also introduces episode level success weighting, which keeps the same data distribution but gives higher loss weight to samples from successful rollouts.

The proposed method is evaluated from task success rate and latency. On the SmolVLA

LIBERO 4-suite evaluation, the original full denoising model, the unweighted two-anchor correction variant, and the weighted two-anchor correction variant achieve average success rates of 74.3%, 74.5%, and 78.0%, respectively. This shows that two-anchor residual correction can reduce denoising calls while maintaining closed loop task success under the current evaluation protocol, and that success weighted training further improves the 4-suite average. Cross seed rechecks using seed 1000, seed 1500, and seed 3000 do not show systematic success rate degradation. In the supplementary $\pi_{0.5}$ experiment, the single anchor residual correction variant obtains a 4-suite average success rate of 89.8%, close to the original model’s 91.1%.

Latency evaluation shows that the speed benefit is not limited to one hardware platform. Under the Ascend 310P dynamic graph setting, π_0 single anchor residual correction reaches 1.44 $\times$ end to end action chunk speedup; SmolVLA two-anchor and single anchor ARD reach 2.00 $\times$ and 2.43 $\times$; and $\pi_{0.5}$ single anchor correction reaches 1.52 $\times$. With the same SmolVLA LIBERO fine tuned model, experiments on Ascend 310P, AGX Orin, and AGX Thor show a cross platform dynamic graph speedup range from 2.00 $\times$ to 4.12 $\times$. Combining cross model and cross platform dynamic graph results, the overall end to end speedup range reported in this thesis is 1.44 $\times$ to 4.12 $\times$. Static graph deployment on Ascend 310P is discussed separately because those results use a different fine tuned model version.

Overall, this thesis provides a practical Ascend adaptation workflow for VLA and embodied policy models, identifies the repeated AE denoising loop as an important optimization target, and proposes ARD as a lightweight residual correction method for continuous action VLA inference. The work also has limitations. The main success rate evidence is based on SmolVLA, while π_0 and $\pi_{0.5}$ mainly provide structure migration, latency accounting, and supplementary validation. On more difficult task combinations such as `libero_10`, the current distillation method remains limited. Future work can study adaptive anchors, failure aware training, automated graph conversion, and real robot tests.